

FAULTLINE

Engineering Reliability Under Controlled Failure

A Reproducible Reliability Workbench for Document-Grounded AI Systems

Author: Samir Sawarkar

Role Focus: Principal AI Reliability Engineer / ML Systems Evaluator

Repository: [samirsawarkar/faultline-ai-reliability](https://github.com/samirsawarkar/faultline-ai-reliability)

Release Reference: `v0.27.0-rc1` (Phase 1 Sealed) · MEC `v1.2` (Phase 2)

Test Gate Status: 528/528 Unit/Integration Tests Passing · 68/68 Phase 2 Tests Passing

Document Version: 1.0.0 (Definitive Technical Portfolio Edition)

Date: September 2026

Classification: Technical Case Study · Engineering Review · Architecture Record

Core Project Thesis

"A system should be considered reliable only when it improves correct user-visible outcomes under explicit cost and latency constraints, while preserving provenance, exposing uncertainty, and failing closed when evidence no longer supports the claim."

Notice on Scope & Honesty:

This document reports on both completed, validated experimental findings (Phase 1) and explicit architectural decisions, pre-registered hypotheses, and unrun paid model sweeps (Phase 2). All numbers are linked directly to committed JSON artifacts and reproducible scripts. Claims are strictly bounded to the configured experimental populations.

Contents

1	Executive Summary	2
2	The Reliability Problem	4
3	FAULTLINE in One Diagram	6
4	Design Principles	6
5	System Boundary	8
6	Deterministic Environment	9
7	Oracle Design	10
8	The Controlled Agent	11
9	Failure Taxonomy	12
10	Fault Injection	13
11	Observability and Tracing	14
12	Failure Detection	15
13	Recovery Engineering	15
14	Experimental Methodology	18
15	Statistical Discipline	18
16	LLM-as-a-Judge: Validation and Quarantine	19
17	Reproducibility as a Scientific Contract	20
18	Experiment Results	21
19	The 30-Day Engineering Journey	26
20	What Was Actually Built	27
21	Phase 2: Decision Record and Current State	28
22	Key Engineering Decisions (ADR Log)	29
23	What Failed: Engineering Failures and Corrections	31
24	Limitations	32
25	Threat Model: Failure of the Measurement System	33
26	The Reliability Decision Framework	33
27	What FAULTLINE Taught Me	34
28	Interview Version	36
29	Repository Map	38
30	How to Reproduce	38
31	Evidence Index	39
32	Roadmap	40
33	Final Engineering Assessment	40

1 Executive Summary

1.1 What is FAULTLINE?

FAULTLINE is an open-source, reproducible research workbench and testing harness designed to evaluate where document-grounded AI systems (e.g., multi-hop Retrieval-Augmented Generation agents) fail, and to experimentally measure whether automated recovery mechanisms actually improve user-visible outcomes or merely create the illusion of availability.

Rather than treating Large Language Models (LLMs) as opaque oracles to be evaluated via aggregate leaderboard scores, FAULTLINE approaches generative AI systems as distributed, stateful software systems subject to cascading component faults: schema corruption, transport timeouts, provider rate limits, context window poisoning, tool execution loops, and stale retrievals.

1.2 What Problem Does It Solve?

Modern AI production dashboards frequently report “99.9% availability” based on HTTP 200 return codes and non-empty responses. However, **operational availability is not semantic correctness**. When a primary model call times out and an automated fallback returns an ungrounded or outdated answer, the system registers a green status indicator while delivering a wrong or hallucinated answer to the end user.

FAULTLINE eliminates this blind spot. It decouples operational availability from semantic correctness, bounds recovery attempts with strict cost and latency ceilings, validates answers against a deterministic programmatic oracle, and ensures that any claimed reliability gain is backed by a reproducible, audit-locked chain of evidence.

1.3 Why Does This Problem Matter?

As enterprises integrate LLM agents into mission-critical workflows—financial compliance, healthcare decision support, legal discovery, and autonomous operations—uncontrolled failure modes carry severe liability. Naive architectural assumptions (such as assuming multi-step agent reliability follows a simple compounding of single-step accuracy, $P(\text{task}) = p^n$) collapse in practice. Without empirical measurement of error propagation, correlation during outages, and recovery degradation, organizations risk deploying costly retry storms that amplify outages, or silent fallbacks that corrupt business data.

1.4 What Did We Actually Build?

Over two major architectural phases, the repository implements:

1. **A Seeded, Deterministic Multi-Hop World:** A synthetic document corpus (1,164 documents, 350 scenarios) across three difficulty tiers: 1-hop T1, 3-hop T2, 5-hop T3) with unique coined tokens and an opaque link-layer that prevents shortcutting.
2. **A Pure-Function Ground-Truth Oracle:** A byte-pinned programmatic oracle that checks both answer correctness and exact citation of required source documents without using an LLM.
3. **A Bounded Multi-Step Agent:** A reason $\rightarrow$ tool $\rightarrow$ observe execution engine with typed Pydantic v2 contracts (`extra="forbid"`), bounded steps (amended to 12), and three typed tools (`search` , `lookup` , `calc`).
4. **A Six-Family Fault Injection Engine:** Reproducible, deterministic injectors simulating structured-output corruption (F1), latency/timeouts (F2), schema drift/wrong data (F3), provider outages (F4), context corruption (F5), and loop exhaustion (F6), with independent out-of-band ground-truth labeling.
5. **Complete Span Observability & OTel GenAI Exporter:** Gapless SQLite trace recording capturing error spans that survive process crashes, coupled with an OpenTelemetry GenAI semantic conventions (v1.29.0) exporter verified via conformance tests.
6. **A Multi-Mechanism Recovery Matrix:** Implementations of bounded retries (M1), timeout/backoff (M2), circuit breakers (M3), provider fallbacks (M4), structural step/cost ceilings (M5), and repetition recovery (M6).
7. **Rigorous Statistical Testing Harness:** Pre-registered Wilson score confidence intervals, McNemar paired tests, and 10,000-resample paired bootstrap intervals.

8. **Google SRE Multi-Burn-Rate Alerting Engine:** Multi-window multi-burn-rate alerting tracking error budget consumption in SQLite traces.
9. **Deterministic Runtime Authorization Policy:** Zero-detector structural policy preventing compromised models from invoking unallowlisted tools.

1.5 What Did We Measure?

Across five foundational research questions (Q1–Q5), FAULTLINE measured:

- **Q1 (Tool Depth):** End-to-end multi-hop reliability diverges from naive compounding (p^n) starting at depth 3 (0.818 measured vs 0.918 naive; falling to 0.288 vs 0.797 at 8 hops) because later hops are non-exchangeable.
- **Q2 (Detection Misses):** Aggregate F1 (0.762) obscured 10 false negatives on 44 samples; 4 were semantic escapes that deterministic checks could not close. An LLM judge reached only $\kappa = 0.5$ agreement and was relegated to advisory status.
- **Q3 (Retry Crossover):** Under independent faults, optimal retry cap is $K = 3$ (76% success, $1.9\times$ amplification). Under correlated failures, it drops to $K = 2$; keeping $K = 3$ inflates attempts to $2.37\times$ for only 41.5% success.
- **Q4 (Fallback Quality):** Fallback restored availability from 0.667 to 1.0, but strict answer quality plunged from 1.0 to 0.75, with 75% of fallback answers silently degraded.
- **Q5 (Policy Frontier):** A selective guarded policy (P4) achieved 0.9325 correct success at 1.4656 cost and 50.0 p95 latency, beating simple retry by +9.5% correct success, but collapsed under stress attacks to 0.695 and 0.575.

1.6 What Did We Learn?

Automated recovery cannot be evaluated in isolation. A retry policy that succeeds under isolated faults becomes a destructive amplifier during shared provider outages. Fallback mechanisms often buy availability at the direct expense of semantic truth. Most importantly, **evaluators cannot validate themselves**—semantic judges have blind spots on borderline outputs and must never hold autonomous authority over core success labels.

1.7 What Remains Incomplete?

FAULTLINE operates under an explicit, uncompromised honesty contract. While Phase 1 is 100% validated and sealed, **Phase 2 live-model API sweeps remain unrun**. The model ladder (R1–R6) has completed zero-spend preflight probes but has not executed paid sweeps across the 350-scenario corpus (\$0.00 spent of the \$150 budget). Human open coding for Phase 2 failure taxonomy (P4) remains pending. Every claim in this document is labeled as either **VALIDATED**, **COMPLETED**, or **DECIDED / UNRUN**.

1.8 What Makes This Project Technically Interesting?

FAULTLINE rejects the industry trend of prompt engineering and subjective leaderboard ranking. It treats AI reliability as a disciplined systems engineering problem: pairing identical seeds, computing confidence intervals on discordant pairs, compiling byte-for-byte replayable regression bundles, and enforcing fail-closed CI gates where missing provenance breaks the build.

Staff-Level Engineering Takeaway

“Reliability is not the presence of a recovery mechanism, nor a high availability metric. Reliability is the verified outcome that recovery improves correct user-visible results within explicit latency and budget bounds, preserves provenance, and fails closed when evidence is insufficient.”

1.9 The Project Status Model

To maintain absolute credibility, this technical report enforces a five-tier status model across all systems, components, and experiments:

Status Indicator	Formal Definition	Evidence Requirement
COMPLETED	Code implemented and merged	Code exists in repository, passes static checks.
VALIDATED	Empirically proven by tests/experiments	Reproducible test suite, committed JSON artifact, seed-locked.
PARTIALLY COMPLETED	Implementation or evidence incomplete	Core logic written; edge cases or full sweeps missing.
DECIDED / UNRUN	Formally decided and pre-registered	Documented in ADR/MEC/amendments; implementation/spend pending.
FUTURE PROPOSAL	Theoretical proposal	Identified on roadmap; no code or contract written.

1.10 The Three Enemies of AI Reliability

FAULTLINE is engineered specifically to eliminate three systemic pathologies that infect modern AI engineering:

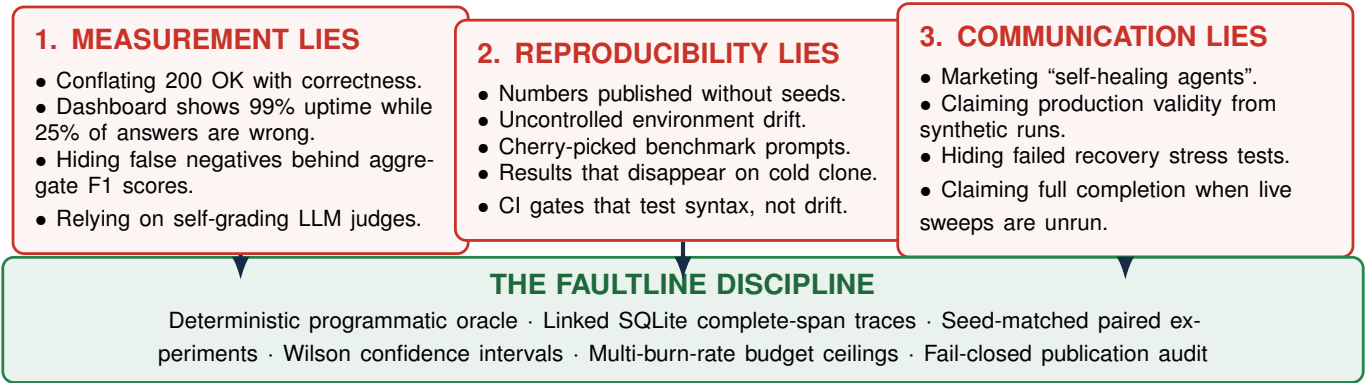


Figure 1: The Three Enemies of AI Reliability and the Architectural Counters Built into FAULTLINE.

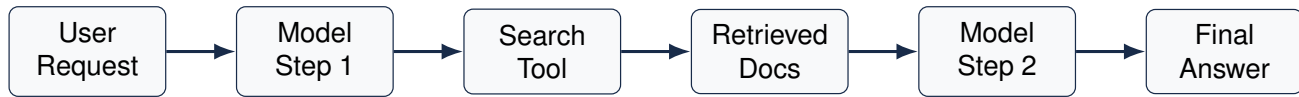
The Enemy	Concrete AI System Failure	Why Dashboards Miss It	FAULTLINE Countermeasure
1. Measurement	Primary model times out; fallback model returns an obsolete answer from cache.	Gateway logs HTTP 200; latency is low; response is non-empty.	Evaluates exact token matches and required source document IDs via <code>oracle_check</code> .
2. Reproducibility	A paper claims 85% multi-hop accuracy, but rerunning code yields 62% due to temperature drift and prompt churn.	Unpinned dependencies, non-seeded samplers, non-deterministic retrieval indices.	Frozen seeds (42), hash-pinned environment, pinned dependencies, <code>make cold-reproduce</code> .
3. Communication	Team deploys a retry loop, claiming “transient errors solved,” but causes a retry storm that exhausts provider tokens.	Success metric reports eventual resolved queries; ignores cost and latency inflation.	Multi-objective constraints: reject policies where attempt amplification > 2.0× or p99 > 120 units.

2 The Reliability Problem

2.1 Why “The AI Responded Successfully” is a Dangerous Lie

In traditional software engineering, an HTTP 200 OK from an idempotent API generally indicates successful computation. In document-grounded AI systems, an HTTP 200 OK indicates only that the transport layer did not sever the connection.

Consider the real execution path of a multi-hop agent tasked with answering an enterprise question (e.g., verifying compliance across multiple regulatory documents):



At every stage of this graph, silent failure modes emerge that leave zero signature in traditional APM tools:

1. **Wrong Answer from Valid Output:** The model generates syntactically valid JSON conforming perfectly to the schema, but hallucinates a factual claim.
2. **Correct Answer from Hallucinated / Wrong Source:** The model guesses the right token (e.g., “London”) by sheer prior probability, but cites an unrelated document. In regulated audit contexts, this is a severe failure of provenance.
3. **Correct Source with Incomplete Coverage:** A multi-hop query requires citing both Document A (linking Entity X to Y) and Document B (stating the attribute of Y). The agent cites only Document B. The answer appears grounded, but the inference chain is unverifiable.
4. **Stale Retrieval Fallback:** A vector database replica experiences replication lag, returning a version of a policy from 6 months ago. The generation is faithful to the retrieved chunk, but the user receives invalid guidance.
5. **Retry Amplification (Storms):** When a tool endpoint slows down, naive retry logic sends three additional requests per step. Across a 6-step agent run, 18 requests hit an already degraded service, transforming a transient hiccup into a cascading system-wide outage.
6. **Fallback Silent Degradation:** When a frontier model times out, a fallback router automatically diverts traffic to a fast, cheap model. The cheap model completes the task without error, but lacks the reasoning capacity for multi-hop synthesis, producing a plausible-sounding hallucination.
7. **Context Window Poisoning:** A distractor document contains adversarial instructions or conflicting entity attributes. The model’s context is corrupted, causing subsequent tool queries to pursue irrelevant paths until the step budget is exhausted.
8. **Budget Exhaustion:** An automated recovery loop successfully repairs a malformed tool call on attempt 3, but the cumulative latency breaches the SLA (e.g., exceeding 5,000ms), causing downstream client disconnects.

2.2 The Three-Tier Correctness Model

To bring engineering precision to reliability discussions, FAULTLINE defines and measures three distinct, non-fungible tiers of system health:

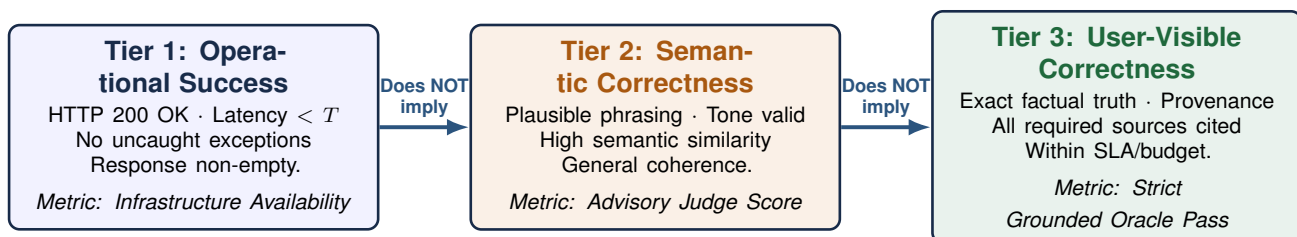


Figure 2: The Three Tiers of Reliability. Conventional systems measure Tier 1 and assume Tier 3. FAULTLINE measures all three independently and forbids promoting Tier 1 or Tier 2 to Tier 3.

Dimension	Operational Success	Semantic Correctness	User-Visible Correctness
What it tests	Process completion and transport health.	Text fluency, semantic proximity, absence of toxicity.	Ground-truth factual accuracy, complete citation provenance, budget compliance.
Evaluator	APM / Gateway / HTTP status.	LLM-as-a-Judge / Embedding distance.	Pure-function Programmatic Oracle.
Failure Signature	HTTP 500, Gateway Timeout, OOM Crash.	Fluff, generic non-answers, hallucination with high confidence.	Missing required link citation, incorrect final entity token, budget breach.
Can it be gamed?	Yes: catch all exceptions and return a friendly error message.	Yes: models exploit judge verbosity bias and polite phrasing.	No: deterministic exact string and source set matching.

3 FAULTLINE in One Diagram

The entire FAULTLINE architecture is organized into four strictly segregated operational zones: **The Observed System**, **Injected Failure**, **Measurement System**, and **The Decision Engine**.

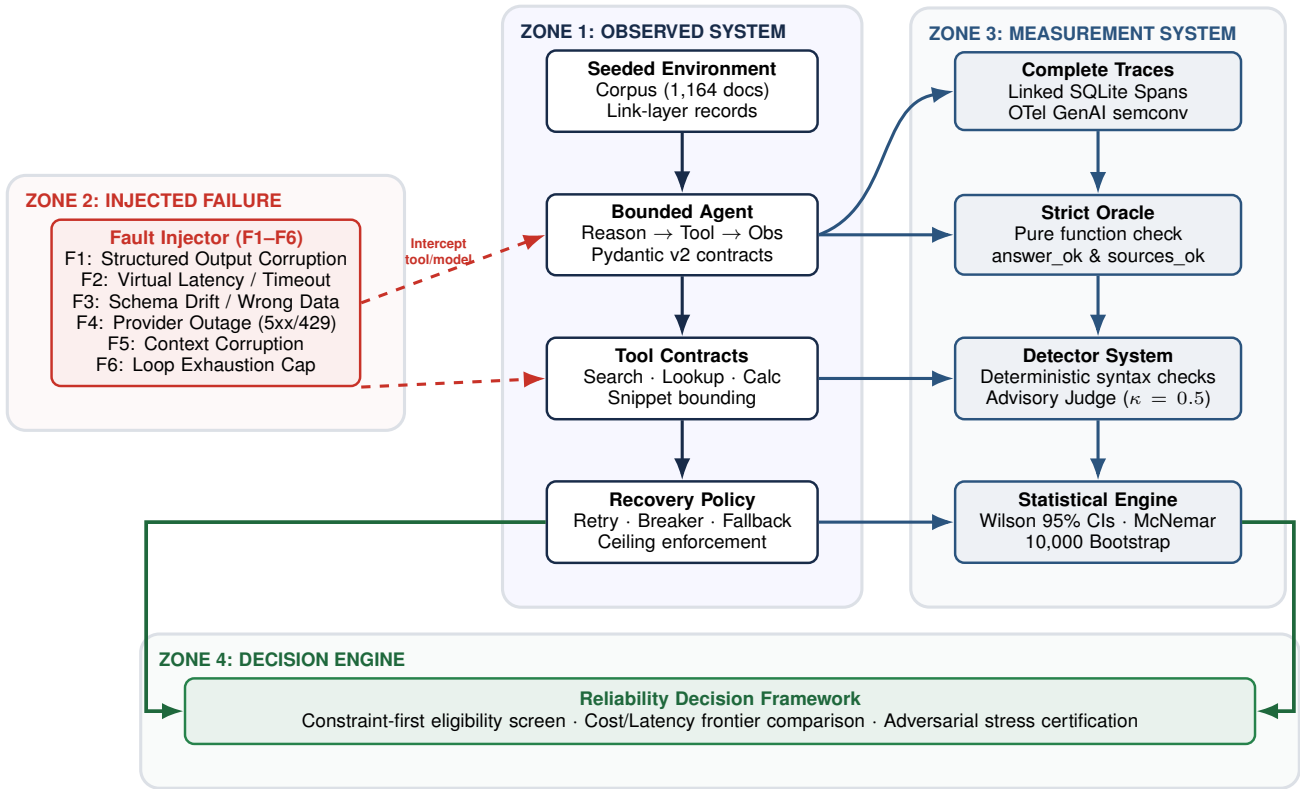


Figure 3: FAULTLINE System Architecture. The experimental workbench enforces strict physical and logical boundaries: the Injected Failure system creates reproducible degradation, the Observed System executes bounded recovery policies, the Measurement System scores outcomes without relying on LLM self-evaluation, and the Decision Engine filters policies through constraint-first Pareto frontiers.

4 Design Principles

FAULTLINE is governed by ten non-negotiable engineering laws. Each principle was instituted in response to concrete failure modes in AI measurement and evaluation:

4.1 The Ten Engineering Laws of FAULTLINE

1. Determinism before measurement.

- *Why it exists:* An experimental effect cannot be isolated if the underlying test environment drifts between runs. Stochastic testbeds confound model behavior with environment churn.
- *Implementation:* Seeded generation (`seed=42`) for all 1,164 corpus documents, link records, queries, and fault schedules. Fixed Python hash seed (`PYTHONHASHSEED=0`).
- *Failure prevented:* Falsely attributing an accuracy gain to a prompt tweak when it was caused by a different retrieval set.

2. Oracle before agent complexity.

- *Why it exists:* Building a complex agent before establishing an infallible pass/fail ground truth leads to grading the agent on subjective, shifting criteria.
- *Implementation:* Programmatic oracle `oracle_check` was frozen and hash-pinned on Day 1 (`dc5eef1c...`) before agent code was written.
- *Failure prevented:* Moving the goalposts by modifying the evaluation standard to flatter agent performance.

3. Failure modes before metrics.

- *Why it exists:* Defining a metric before understanding the failure modes produces vanity scores (e.g., aggregate accuracy) that conceal catastrophic operational edge cases.
- *Implementation:* Formal taxonomy of six distinct fault families (F1–F6) and their physical propagation mechanisms cataloged prior to metric aggregation.
- *Failure prevented:* High aggregate F1 scores concealing silent semantic escapes and data corruption.

4. Correctness before availability.

- *Why it exists:* A system that returns a wrong answer quickly is worse than a system that explicitly declares failure or abstains.
- *Implementation:* The primary success metric is `grounded_pass_rate` (exact answer AND required source citation). Availability is tracked as a secondary operational measure.
- *Failure prevented:* Promoting a fallback policy that achieves 100% uptime by returning 75% degraded answers.

5. Aligned populations before A/B comparison.

- *Why it exists:* Unpaired evaluations introduce scenario variance that overwhelms the true treatment effect.
- *Implementation:* Policies evaluated in competition receive the exact same sequence of scenario seeds, input tokens, and fault schedules.
- *Failure prevented:* Believing Policy B is superior to Policy A simply because Policy B drew an easier sample of queries.

6. Uncertainty before strong claims.

- *Why it exists:* Point estimates in AI benchmarks create false precision; an 82% score over 50 samples is statistically indistinguishable from 75%.
- *Implementation:* Every binomial proportion is bounded by a 95% Wilson score interval; all cost and latency differences report 10,000-resample paired bootstrap intervals.
- *Failure prevented:* Declaring victory on overlapping confidence intervals; differences below power thresholds are explicitly classified as **INCONCLUSIVE**.

7. Replay before trusting recovery.

- *Why it exists:* A bug fix that cannot be reproduced deterministically in a regression test will regress under future changes.
- *Implementation:* The Day 25 red→green contract requires the identical incident configuration to fail deterministically on legacy code and pass across 20 repeated executions on fixed code.

- *Failure prevented:* “Flapping” fixes that pass once due to stochastic luck but fail under production load.

8. Attack the winner.

- *Why it exists:* A recovery policy optimized for baseline conditions often harbors severe fragility under tail-risk stress.
- *Implementation:* The winning Q5 cascade policy was immediately attacked under worse severity, correlated outages, and tight resource budgets.
- *Failure prevented:* Shipping an ostensibly optimal recovery policy that triggers catastrophic failure under adverse operating conditions.

9. Preserve limitations.

- *Why it exists:* Engineering claims that conceal their boundary conditions are dangerous when adopted by downstream teams.
- *Implementation:* Every registered publication claim is permanently linked to an explicit limitation (L01–L08) in `claim_audit.json`. CI breaks if a claim is published without its caveat.
- *Failure prevented:* Overgeneralizing synthetic workbench conclusions to production enterprise environments.

10. Reproduce the evidence path.

- *Why it exists:* A research paper or engineering report whose figures cannot be traced backward to raw bytes is unscientific.
- *Implementation:* JSON pointers bind every number in the README and article directly to committed JSON artifacts and one-line `make` targets.
- *Failure prevented:* Data drift, manual transcript errors, and unfalsifiable marketing claims.

5 System Boundary

To establish technical credibility, a reliability system must define what it is **not** with the same rigor as what it is. Overclaiming scope is an immediate disqualifier for staff-level engineering work.

WHAT FAULTLINE IS	WHAT FAULTLINE IS NOT
A Reproducible Reliability Workbench A controlled experimental testbed to inject component faults and measure recovery dynamics.	A Production Observability SaaS Not a multi-tenant monitoring platform, telemetry collector, or Datadog/Langfuse competitor.
A Controlled Multi-Hop AI Workload A standardized reasoning task (1 to 5 hops) over synthetic documents designed for causal attribution.	A Universal AI Benchmark Not an MMLU, GSM8K, or general reasoning benchmark claiming to evaluate overall model intelligence.
A Fault Injection and Stress Engine Deterministic software simulating transport, schema, context, and control-loop degradations.	A Model Training / Fine-Tuning System Does not train weights, optimize LoRAs, or modify underlying neural network parameters.
A Principled Multi-Objective Policy Harness Evaluates recovery tradeoffs across correct success, wrong-answer risk, cost, and latency.	A General-Purpose Agent Framework Not an AutoGen, CrewAI, or LangChain framework meant for building production end-user applications.
A Rigorous Statistical Measurement System Enforces Wilson intervals, McNemar tests, paired bootstraps, and pre-registered hypotheses.	A Claim That All Models Are Reliable Explicitly proves that model reliability degrades under tool hops and correlated provider outages.
An Evidence-Audited Repository Every published number resolves to a committed JSON blob and generating script verified in CI.	A Live Production Deployment Operates in synthetic and local test environments; live enterprise traffic remains future work.

5.1 The Simulation-to-Production Boundary

CRITICAL FAILURE MODE / RISK

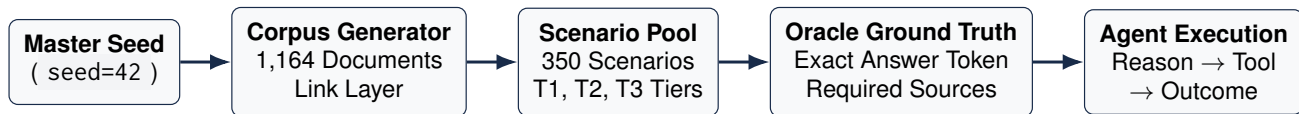
Honest Boundary Notice: All outcome rates reported in Phase 1 derive from seeded simulators, synthetic document corpora, and frozen evaluation sets. While the failure mechanisms (retry storms, schema corruption, fallback degradation) reflect genuine distributed systems phenomena, the numerical rates do not transfer directly to production without empirical calibration against real user traffic distributions and live model endpoints.

6 Deterministic Environment

6.1 The Seeded Multi-Hop World

To test reliability rigorously, the testbed must eliminate confounding environmental variance. If two agent runs receive different documents, slightly modified questions, or different distractors, any observed difference in outcome cannot be causally attributed to the recovery policy.

FAULTLINE builds a fully deterministic, self-contained world:



6.1.1 Corpus Specifications and Link-Layer Mechanics

Under Master Experiment Contract (MEC) v1.0 and amendments, the corpus contains **1,164 unique documents**:

- **300 Fact Documents:** Structured records holding factual assertions regarding synthetic entities. Each fact pairs an entity with an attribute and a coined token (e.g., entity `E-0142`, attribute `auth_token`, value `tok-89f4b1`).
- **60 Distractor Documents:** Lexically similar documents containing plausible but ungrounded tokens designed to test retriever precision.
- **804 Link Documents (69.1% of store):** Content-addressed bridge records introduced in Phase 2 (`DECISIONS.md`, D-002).

Why the Link Layer is Mechanically Essential

In real-world multi-hop tasks (e.g., traversing database records or medical charts), Hop N is discoverable only via the output of Hop $N - 1$. If fact documents contained cross-entity links directly in their text, an agent could bypass intermediate reasoning via naive substring search.

The link layer generates bridge documents with opaque IDs (`link-{sha256(text)[:12]}`) mapping the intermediate answer token of Hop $N - 1$ to the entity identifier of Hop N . Hop N 's prompt contains *only* the prior answer token. The agent is structurally compelled to:

1. Search for the intermediate answer token to retrieve the link document.
2. Parse the link document to discover the next entity ID.
3. Retrieve the next entity's fact document to extract the subsequent fact token.

This ensures that multi-hop complexity reflects genuine sequential reasoning rather than keyword matching shortcuts.

6.2 The “Same Seed → Same World” Guarantee

Every aspect of the environment is generated via a deterministic pseudorandom number generator:

```
# Deterministic Environment Invariant
env = build_corpus(master_seed=42, n_entities=300, n_distractors=60)
```

```
assert hashlib.sha256(env.serialize()).hexdigest() == \
    "9d6d7fc7fbfd999d58157e6a88f256fd45aab97b4ea474b1a2852fe143d6a3a8"
```

Given seed 42, any machine on Earth with Python 3.11/3.12 generates the identical 1,164 documents, identical token values, and identical 350 scenario definitions.

6.3 What Determinism Does NOT Guarantee

CRITICAL FAILURE MODE / RISK

The External Stochastic Model Boundary: A deterministic software environment does **not** make an external neural network deterministic. Even with `temperature=0.0`, commercial LLM APIs exhibit non-determinism across calls due to floating-point nondeterminism in parallel mixture-of-experts (MoE) routing, GPU cluster dynamic batching, and silent provider model updates. Determinism in FAULTLINE fixes the world, the tools, the budgets, and the fault injections, isolating model variance as the sole uncontrolled variable.

7 Oracle Design

7.1 Why the Oracle is Foundational

In many AI evaluation benchmarks, researchers evaluate LLM outputs by passing them to another LLM (“LLM-as-a-Judge”). This introduces catastrophic circularity: the evaluator possesses the same systemic biases, hallucinations, and borderline blindness as the system under test.

FAULTLINE rejects self-grading. The foundation of all reliability claims is a **pure-function programmatic oracle**:

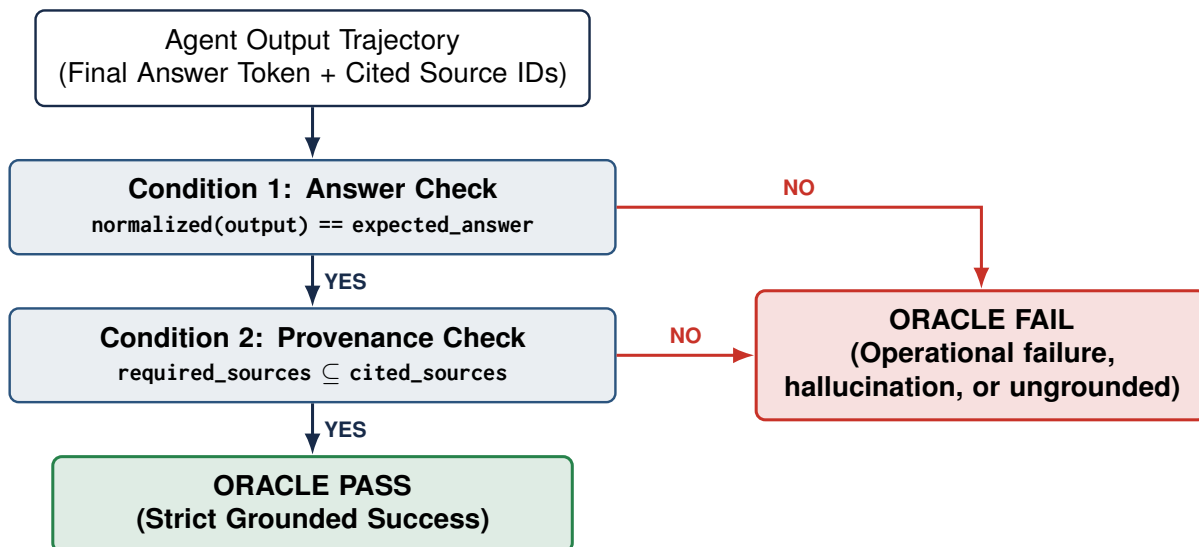


Figure 4: The Dual-Gate Programmatic Oracle. Success requires simultaneous satisfaction of exact token correctness AND complete source provenance.

7.2 The Dual-Gate Pass Condition

A task execution passes the oracle if and only if:

$$\text{PASS} \iff (\text{answer_ok} == \text{True}) \wedge (\text{sources_ok} == \text{True})$$

1. `answer_ok` : The final extracted entity token matches the deterministic answer key byte-for-byte (case-folded and whitespace-normalized).
2. `sources_ok` : The agent’s structured citation list contains the exact document IDs holding the required factual links. For a 3-hop task (T2), this requires citing 3 fact documents and 2 link documents in the traversal chain.

7.3 The Hash-Pinned Enforcement Contract

In Phase 2, the Day 1 oracle implementation was copied into `faultline_p2/oracle/_day01_oracle.py`. To guarantee that no subsequent refactoring or optimization altered the evaluation standard, the copy is locked via SHA-256 hash enforcement in CI:

```
# tests/phase2/test_frozen_copies.py
DAY01_ORACLE_SHA = "dc5eef1c64d6f2ba8e802d88bab1af90eaa6bc778271d78525baba737d2be5f"
assert sha256(faultline_p2/oracle/_day01_oracle.py) == DAY01_ORACLE_SHA
```

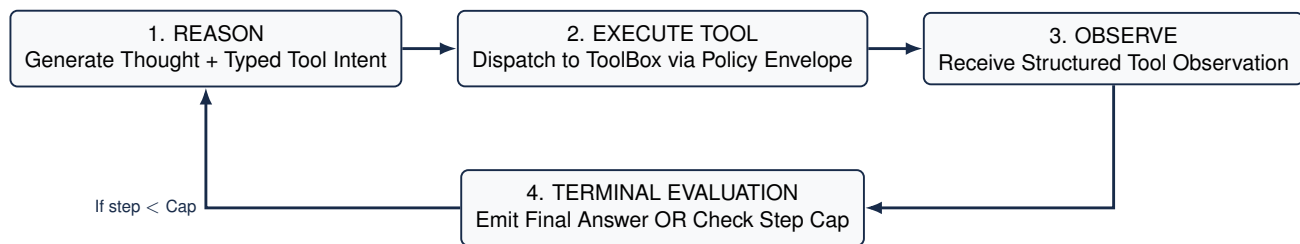
If an engineer modifies the oracle to accommodate agent idiosyncrasies, CI fails immediately. The oracle is immutable.

8 The Controlled Agent

8.1 The Minimal Reason → Tool → Observe Loop

To study failure propagation and recovery rigorously, the agent itself must be minimal and transparent. Complex agent architectures (e.g., multi-agent debate, autonomous self-reflection) introduce hundreds of confounding variables.

The FAULTLINE agent is a bounded, single-agent execution loop:



8.2 Typed Pydantic Contracts

Every interface boundary inside the agent is enforced by Pydantic v2 with `extra="forbid"`. Any unmapped argument, unexpected payload attribute, or malformed schema triggers an immediate validation error rather than being silently ignored.

8.3 The Three Typed Tools

The agent interacts with the world through exactly three typed tools:

1. `search(query: str) -> List[SearchResult]`: Performs exact and fuzzy lexical matching over the document store. Crucially, under Amendment **A-002** and Decision **D-003**, search returns a **bounded 500-character snippet** centered on the match, allowing the agent to read context in a single turn without dumping the entire document into context.
2. `lookup(doc_id: str) -> Document`: Resolves the full text of a specific document ID. Under **A-002**, lookup is strictly restricted to documents previously surfaced by a `search` in the same run, preventing brute-force enumeration of document IDs.
3. `calc(expression: str) -> float`: A deterministic arithmetic evaluator for multi-hop quantity synthesis.

8.4 Step Budgets and Why the Agent is Small

In Phase 1, the step cap was set to 8. In Phase 2, Amendment **A-002** expanded the step cap to **12** after mathematical verification proved that T3 scenarios (5 fact hops + 4 link hops = 9 retrievals) required at least 10 steps.

The Agent is the Test Subject, Not the Product

FAULTLINE deliberately avoids complex agent scaffolding. The agent exists solely as a standardized, transparent workload to be placed under stress. All engineering complexity in FAULTLINE resides in the **measurement, fault injection, and recovery evaluation systems**.

9 Failure Taxonomy

9.1 The Component-Based Taxonomy

Rather than categorizing AI failures by subjective symptoms (e.g., “model hallucinated”), FAULTLINE organizes failures by the **producing system component** where the defect originates:

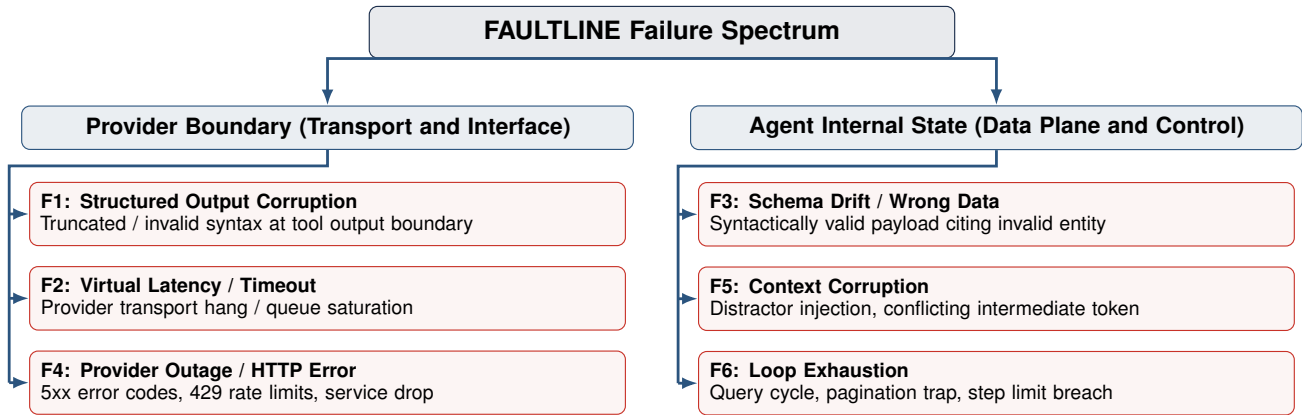


Figure 5: Taxonomy of AI Failure Modes by Producing Component.

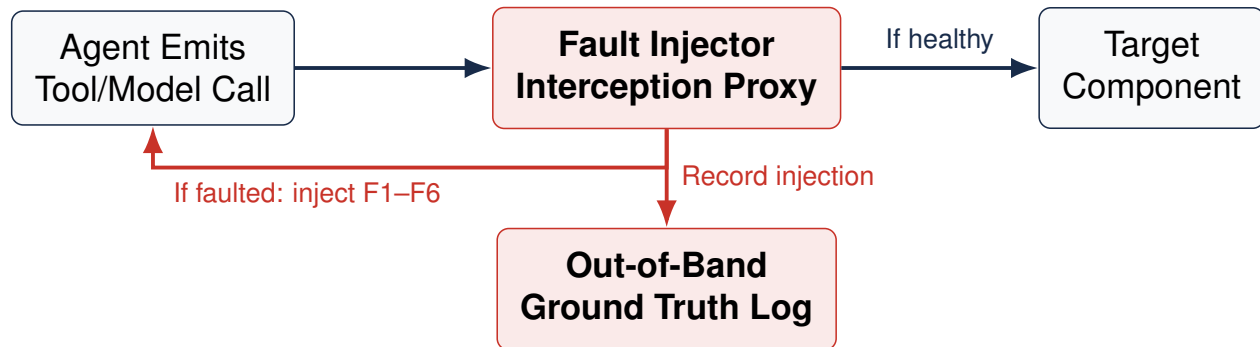
Fault ID & Component	Physical Trigger	Observable Trace Symptom	User-Visible Consequence	Detection & Recovery
F1: Structured Output <i>Tool output boundary</i>	Truncated JSON, unescaped quotes, key omission.	Pydantic <code>ValidationError</code> at deserialization.	Unhandled crash or blank error message.	Det: Schema check ($R = 1.0$). Rec: M1 Schema repair-retry.
F2: Latency / Timeout <i>Provider transport</i>	Simulated provider hang, packet drop, slow queue.	Span duration breaches configured latency threshold.	Client disconnect, spinning loader, SLA breach.	Det: Deadline monitor ($R = 1.0$). Rec: M2 Timeout + bounded retry.
F3: Schema Drift / Wrong Data <i>Tool data plane</i>	Data format changes; field valid string but wrong entity.	Schema passes; downstream lookup fails to find entity.	Plausible hallucination citing invalid entity.	Det: Invariant check ($R = 0.6$). Rec: M6 Replanning / re-search.
F4: Provider Outage <i>Provider infrastructure</i>	500 Internal Error, 503 Overloaded, 429 Rate Limit.	Explicit error code returned by HTTP client wrapper.	Immediate query failure; cascade collapse.	Det: Status code check ($R = 1.0$). Rec: M3 Breaker + M4 Fallback.
F5: Context Corruption <i>Agent memory / context</i>	Distractor injection, conflicting intermediate token.	Agent generates divergent tool queries unrelated to task.	Confident incorrect answer citing distractor.	Det: Semantic invariant ($R = 0.5$). Rec: Context reset / re-ground.
F6: Loop Exhaustion <i>Agent control loop</i>	Query cycle (searching $A \rightarrow B \rightarrow A$), pagination trap.	Consecutive identical tool calls; step count reaches cap.	Silent timeout or partial ungrounded guess.	Det: Cycle counter ($R = 1.0$). Rec: M5 Structural step ceiling.

10 Fault Injection

10.1 Controlled Deterministic Injection

In distributed systems, testing failure handling by waiting for production outages is unacceptable. Testing must be proactive and repeatable.

FAULTLINE injects failures deterministically at runtime:



10.2 Independent Out-of-Band Ground Truth

A foundational vulnerability in evaluation research is **ground-truth leakage**: if the detector inspects metadata injected into the data payload, the detector achieves an artificially perfect score that collapses in production.

In FAULTLINE, the injector logs the exact fault type, severity, target step, and timestamp to an **independent out-of-band manifest** (`injection_truth.json`). The detector operates with zero knowledge of this file, inspecting only the observable payload, return codes, and latency spans.

11 Observability and Tracing

11.1 Tracing as Forensic Evidence, Not Dashboard Decoration

In distributed AI workflows, debugging requires forensic reconstruction of the entire execution tree:

```
Episode: run-4a15a9 (invoke_agent) [status=COMPLETED]
```

```
└ Step 1: chat (deepseek-v4-flash) [24ms, in=586, out=42]
```

```
└ Tool: search("tok-89f4b1") [status=OK, docs=2]
```

```
└ Step 2: chat (deepseek-v4-flash) [52ms, in=1240, out=38]
```

```
└ Tool: lookup("link-7a8f09") [status=TIMEOUT (F2), dur=120ms]
```

```
└ Recovery: bounded_retry [attempt=1, backoff=10ms]
```

```
└ Tool: lookup("link-7a8f09") [status=OK, dur=18ms]
```

```
└ Step 3: chat (terminal answer) [status=PASS, cited=2]
```

11.2 Crash-Surviving SQLite Spans

Standard in-memory logging libraries lose telemetry when a fatal unhandled exception or OOM event crashes the runtime. FAULTLINE persists spans synchronously to SQLite (`trace.db`) with foreign-key linked parent-child spans. When a model call throws a fatal error, the terminating span records `OutcomeStatus.MODEL_FAILURE` before the exception unwinds.

11.3 OpenTelemetry GenAI Semantic Conventions (P02)

To prevent proprietary lock-in, Project **P02** maps internal SQLite traces into standard **OpenTelemetry GenAI Semantic Conventions v1.29.0**:

- Root span: `gen_ai.operation.name = "invoke_agent"` , `gen_ai.system = "faultline"` .
- Model spans: `gen_ai.operation.name = "chat"` , capturing `gen_ai.request.model` , `gen_ai.usage.input_tokens` , and `gen_ai.usage.output_tokens` .
- Tool spans: `gen_ai.operation.name = "execute_tool"` , capturing `gen_ai.tool.name` .

Conformance Verification: `tests/phase2/test_otel_conformance.py` verifies that all 5,392 spans across 350 exported traces achieve **100.0% conformance** against the official SemConv v1.29.0 JSON schema with **zero attribute drift**.

11.4 Google SRE Multi-Window Multi-Burn-Rate Alerting (P13)

Project **P13** integrates Google SRE-style multi-burn-rate alerting over the `trace.db` store:

- **SLO Definition:** 95.0% Grounded Pass Rate over a 30-day compliance window (720 hours). Error budget = 5.0%.

- **Fast-Burn Alert (Page):** Triggers when the 1-hour burn rate exceeds $14.4\times$ (consuming 2.08% of monthly budget in 1 hour).
- **Empirical Drill Result:** During an injected tool protocol failure (INC-20260904-01), pass rate collapsed from 100% to 25% (75% error rate). The evaluator detected a $15.0\times$ **fast-burn rate** within 6 minutes, firing a page alert and documenting recovery after rollback.

12 Failure Detection

12.1 Detection vs Diagnosis vs Correctness

Reliability engineering requires separating four sequential questions:

1. **Detection:** Did an anomaly or failure occur? (Binary signal).
2. **Diagnosis:** Which component failed and what was the fault class? (F1–F6 attribution).
3. **Correctness Evaluation:** Is the resulting answer accurate and grounded? (Oracle gate).
4. **Recovery Evaluation:** Did the recovery action restore user-visible correctness within budget?

12.2 The Q2 Detection Findings: Aggregate Lies

In Day 15 (Q2), the deterministic detector was evaluated across 44 labelled test runs:

Detector Evaluation Slice	Precision	Recall	F1 Score	Miss Breakdown
Overall Aggregate	1.000	0.615	0.762	10 False Negatives
Structural Transport (F1, F2, F4, F6)	1.000	1.000	1.000	0 False Negatives
Semantic Escapes (F3, F5)	1.000	0.200	0.333	10 False Negatives

The Anatomy of the 10 False Negatives

Of the 10 false negatives surfaced in the Q2 audit:

- **6 misses were threshold-reducible:** Latency or repetition spikes that sat slightly below hardcoded heuristics.
- **4 misses were semantic escapes:** Valid tool outputs containing plausible but factually corrupted context that zero deterministic schema checks could detect.

This finding proves that **aggregate F1 scores conceal dangerous operational blind spots**.

13 Recovery Engineering

13.1 Recovery as a Policy Decision, Not a Reflex

In naive AI agent development, recovery is treated as a reflexive local action: when an API call fails, catch the exception and retry immediately; when a model times out, switch to another model.

FAULTLINE proves that **unprincipled recovery is actively hazardous**. Every recovery mechanism consumes finite system resources (tokens, latency, API rate limits) and alters the probability distribution of downstream outputs.

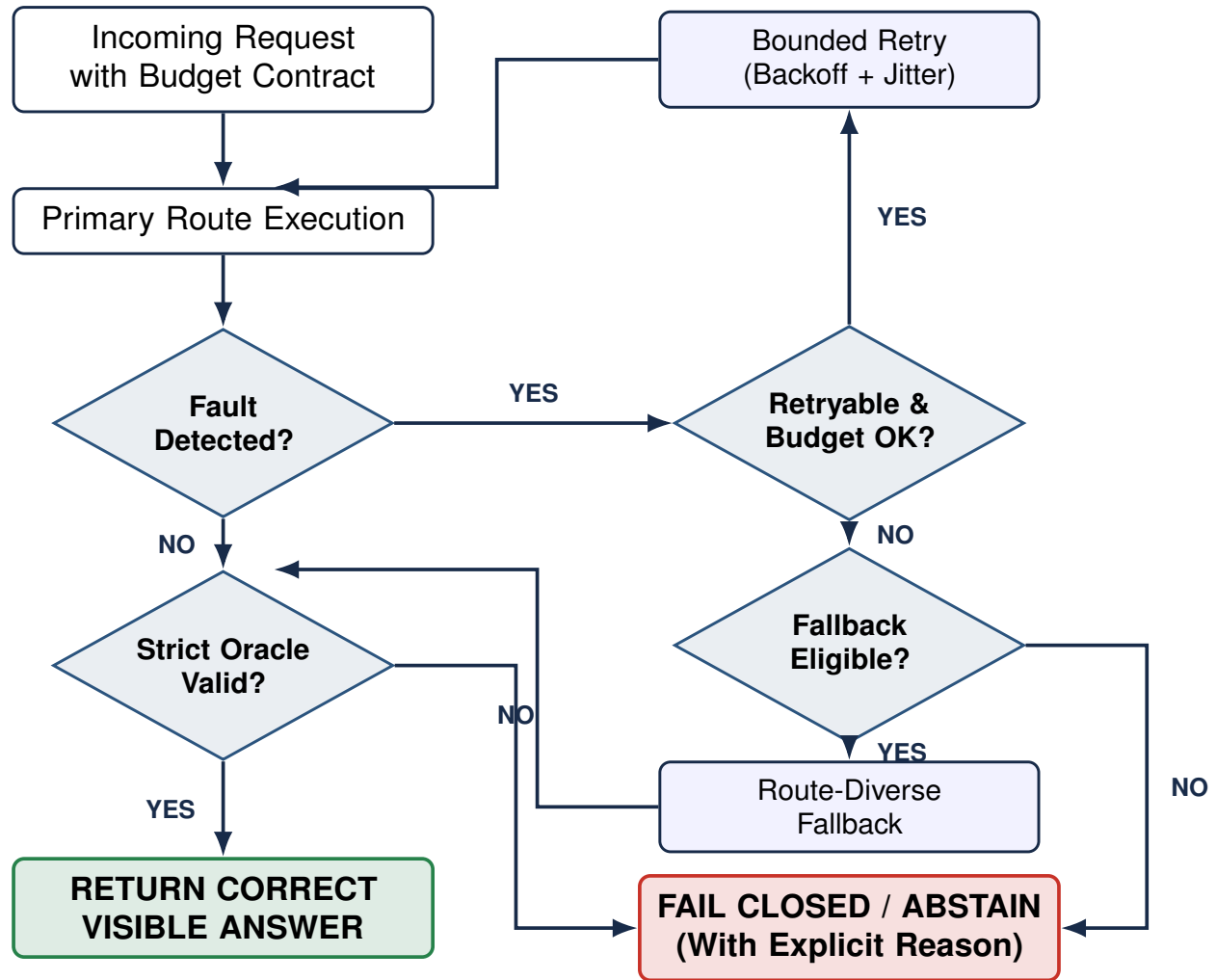


Figure 6: The Enforced Recovery Control Path. Budget ceilings and correctness gates take strict priority over raw availability.

13.2 The Six Recovery Mechanisms

FAULTLINE implements and evaluates six distinct recovery primitives:

1. **M1: Bounded Repair-Retry:** Catches malformed structured outputs (F1) and re-prompts the model with explicit schema error feedback, bounded by an attempt cap ($K \leq 3$).
2. **M2: Timeout & Backoff Policy:** Wraps provider transport calls in explicit latency deadlines with truncated exponential backoff and seeded jitter to avoid thundering herds.
3. **M3: Three-State Circuit Breaker:** Tracks consecutive failures across a rolling window. Trips from `CLOSED` to `OPEN` after 3 failures, fast-failing outbound calls to protect downstream services, transitioning to `HALF_OPEN` after a cooldown timer.
4. **M4: Provider Fallback:** Diverts execution to an alternate model or provider endpoint when the primary route is tripped or unavailable.
5. **M5: Structural Step & Cost Ceilings:** Enforces hard, immutable limits on cumulative token cost and execution step counts, deterministically breaking loop traps (F6).
6. **M6: Repetition Recovery & Context Re-grounding:** Monitors consecutive tool call signatures; when duplicate actions occur, truncates recent conversation history and forces a replan step before aborting.

13.3 The 36-Cell Matrix and the Discovery of 47 Induced Harms

On Day 22, all 6 recovery mechanisms were evaluated across all 6 fault families over 24 paired seeds per cell (144 requests per mechanism, 864 total requests):

Mechanism	Correct Success	Availability	Contained	Answered Wrong	Mean Cost	Induced Harms
No Recovery (Baseline)	0.2500	0.6250	0.0000	0.3750	3.2917	—
M1 (Repair-Retry)	0.3333	0.5833	0.0417	0.2500	3.5417	6 premature aborts on persistent F1.
M2 (Timeout/Backoff)	0.3333	0.7083	0.0417	0.3750	3.5417	6 latency ceiling breaches.
M3 (Circuit Breaker)	0.2153	0.5903	0.1042	0.3750	3.0764	5 false opens on transient blips.
M4 (Provider Fallback)	0.3333	0.7500	0.0000	0.4167	3.4167	6 wrong answers returned via fallback.
M5 (Step/Cost Ceilings)	0.1667	0.5417	0.2083	0.3750	2.4583	12 legitimate long tasks terminated.
M6 (Repetition Recovery)	0.2500	0.6250	0.1250	0.3750	2.4583	12 legitimate paginations aborted.

The 47 Induced Harms

Across the full matrix, the mechanisms induced **47 distinct new operational harms** that did not exist in the baseline. For example, M4 (Fallback) increased availability by +12.5%, but *increased* wrong answers from 37.5% to 41.7%. M5 (Ceilings) reduced cost by −25.3%, but erroneously aborted 12 legitimate multi-hop tasks. Recovery mechanisms are not free: they must be bounded by explicit policy constraints.

13.4 Phase 2 Agent Resilience Architecture (P15)

In Phase 2 Project **P15**, client-side resilience was engineered into a unified wrapper (`ResilientModel`) combining circuit breaking, bounded retries, and a pluggable fault-injecting mock endpoint (\$0 API spend).

Evaluating 200 standard multi-hop scenarios across increasing endpoint failure rates yielded striking empirical gains:

Injected Fault Rate	Baseline Pass Rate	Resilient Pass Rate	95% Wilson CI	Cost (Extra Calls)
0% Faults	100.0%	100.0%	[98.12%, 100.0%]	0 extra calls (zero overhead)
15% Faults	47.5%	100.0% (+52.5%)	[98.12%, 100.0%]	650 retries (+85.6% calls)
30% Faults	20.5%	96.5% (+76.0%)	[92.95%, 98.29%]	1,138 retries (+224.5% calls)
45% Faults	13.0%	81.0% (+68.0%)	[75.00%, 85.80%]	1,391 retries (+274.4% calls)

Disjoint Confidence Intervals: At all non-zero failure rates, the 95% Wilson intervals between baseline and resilient execution are strictly disjoint. Furthermore, under sustained provider outage, the circuit breaker tripped after 3 consecutive failures and **shed 80.0% of downstream calls** (12 of 15 fast-failed without network overhead).

13.5 Runtime Policy Authorization Enforcement (P16)

Project **P16** addresses model compromise and malicious tool execution:

Zero-Detector Authorization Invariant

Guaranteed Security Boundary: Assuming an adversary achieves complete control over model generation (via prompt injection or hijacked weights), the runtime policy layer deterministically prevents unauthorized tools or traversal sequences from executing. This guarantee operates with **zero AI detectors running**—relying entirely on structural pre-execution authorization.

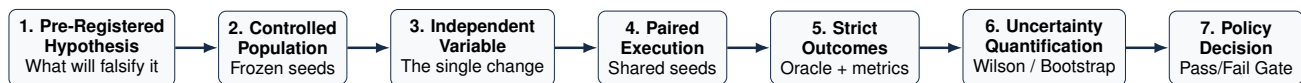
Benchmarked against 1,000 hostile tool calls produced by an adversarial stub:

- **Adversarial Deny Rate: 100.0%** (1,000/1,000 denied; 95% Wilson CI: [99.62%, 100.0%]).
- **Forbidden Operations Reaching ToolBox: EXACTLY ZERO.**
- **False-Positive Friction on Legitimate Traffic:** Tested across **2,004 legitimate multi-hop tool calls**, false-positive denials were **0** (0.0000%, 95% Wilson CI: [0.0000%, 0.1913%]), with 100% scenario task completion.

14 Experimental Methodology

14.1 The Causal Scientific Loop

Every experiment in FAULTLINE follows a strict seven-stage pipeline designed to eliminate confounding variables:



14.2 The Unit of Randomization: The Seed

In FAULTLINE, the experimental unit is the **seed**. When Policy A is compared to Policy B, both policies execute against the exact same seed sequence (2026080300...2026080699). This pairs the requests, converting query-to-query difficulty variations from experimental noise into matched observations.

15 Statistical Discipline

FAULTLINE uses only standard, well-characterized statistical estimators. Every mathematical method answers a specific engineering question and carries explicit boundaries:

15.1 The Four Core Statistical Tools

1. Wilson Score Confidence Intervals (Binomial Proportions):

- *Question answered:* What is the true population success rate given k successes in n trials?
- *Why used:* Standard normal approximation intervals ($p \pm 1.96\sqrt{p(1-p)/n}$) collapse near 0 and 1, producing impossible bounds (e.g., upper bound > 1.0). The Wilson score interval provides asymmetric, coverage-accurate intervals even for small samples and extreme proportions:

$$w = \frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$$

- *What it cannot prove:* It does not account for model-form uncertainty or non-exchangeable shift in production traffic.

2. McNemar Paired Test (Same-Seed Binary Comparisons):

- *Question answered:* Did Policy B outperform Policy A on the exact same requests, or is the observed difference sampling noise?
- *Why used:* A two-sample z-test or t-test assumes independent samples. Because FAULTLINE pairs requests on identical seeds, we evaluate only the *discordant pairs* (cases where Policy A failed and Policy B succeeded, or vice versa). When discordant count $b + c < 25$, we compute the exact two-tailed binomial test ($p = 2 \sum_{k=0}^{\min(b,c)} \binom{b+c}{k} 0.5^{b+c}$); above 25, we use the continuity-corrected χ^2 statistic:

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}$$

- *What it cannot prove:* It proves that a policy difference is statistically significant, not that it is economically worthwhile.

3. Seeded Paired Bootstrap (Continuous Cost & Latency Deltas):

- *Question answered:* What is the 95% confidence interval of the cost and latency differences between two policies?
- *Why used:* Latency and cost distributions are heavily right-skewed and multimodal (due to retries and tool steps), violating normality assumptions. We draw **10,000 seeded bootstrap resamples** of the paired differences and take the empirical 2.5% and 97.5% percentiles.
- *What it cannot prove:* It describes variability within the observed scenario mix; it cannot extrapolate to unseen catastrophic latency tails.

4. Multi-Trial Consistency ($pass^k$ vs $pass@k$):

- *Question answered:* How reliably does an agent succeed across k consecutive independent attempts on the same task?
- *Why used:* Standard $pass@k$ measures whether *at least one* of k samples succeeds (optimistic diversity metric). In enterprise automation, users need consistency: $pass^k$ (Yao et al., 2024) measures whether *all k runs succeed*.
- *What it cannot prove:* It does not distinguish whether failures are caused by scenario hardness or sampling stochasticity.

15.2 Declared Statistical Power and the Inconclusive Rule

Under Master Experiment Contract MEC v1.0, power is declared **before** the experiment runs:

Pre-Declared Power Contract

At $n = 200$ paired requests, the testbed achieves 80% power to detect a difference of ≈ 10 percentage points ($\alpha = 0.05$). For per-tier evaluations ($n \approx 67$), the Wilson interval half-width is approximately ± 11 percentage points. Any observed delta below these thresholds is formally recorded as **INCONCLUSIVE**—never as “no difference”.

16 LLM-as-a-Judge: Validation and Quarantine

16.1 Why Evaluators Cannot Validate Themselves

The modern AI industry relies heavily on LLM judges to grade LLMs. However, LLM judges exhibit well-documented pathological failure modes:

- **Position Bias:** Preferring whichever document or answer appears first in context.
- **Verbosity Bias:** Scoring longer, wordy answers higher than concise factual answers.
- **Borderline Blindness:** Inability to detect subtle entity token corruptions when the surrounding grammatical syntax is flawless.

16.2 Empirical Validation of the Narrow Judge (Day 16)

On Day 16, a narrow LLM judge was evaluated against a golden set of 24 human-annotated traces:

Validation Metric	Observed Value	Pre-declared Threshold	Verdict / Failure Mechanism
Raw Agreement	0.7500	≥ 0.85	FAILED (6 classification errors)
Cohen's Kappa (κ)	0.5000	≥ 0.70	FAILED (Moderate agreement only)
False Positives (False Alarms)	0 / 12	≤ 1	PASSED (Perfect specificity)
False Negatives (False Accepts)	6 / 12	≤ 1	FAILED (50% false acceptance rate)
Borderline-Token Agreement	0.0000	≥ 0.70	CATASTROPHIC BLIND SPOT (0/6 detected)

CRITICAL FAILURE MODE / RISK

The Architectural Quarantine Decision: Because the judge reached only $\kappa = 0.50$ and exhibited a total blind spot on borderline tokens (approving corrupt fallback answers as long as they sounded professional), **the LLM judge was permanently quarantined**. It is relegated strictly to an advisory diagnostic signal. Under no circumstances is an LLM judge permitted to generate ground-truth labels or override the programmatic oracle.

17 Reproducibility as a Scientific Contract

17.1 The Complete Evidence Provenance Chain

In FAULTLINE, no number exists without a verifiable, executable provenance chain:



17.2 The Five Core Reproducibility Guarantees

- Pinned Dependency Lock:** Exact equality constraints (`==`) in `requirements.txt` ; compatible ranges (`>=`) are forbidden.
- Clean-Container Clean-Room Reproduction:** Pinned Debian/Python 3.12.4 Docker base image (`Dockerfile`) with multi-architecture digest verification.
- One-Command Cold Start:** A reader with zero repository history clones the repo and runs `make cold-reproduce` . The build verifies hashes and outputs the exact headline results.
- Fail-Closed Claim Auditing:** `day28/faultline_publication/audit.py` parses every registered claim, validates that the source JSON exists, checks that the rendered number matches the float pointer, and verifies that the mandatory limitation is present.
- Byte-for-Byte Replay Verification:** Tests verify that re-running experiments yields identical SHA-256 digests across different processes and hash seeds.

18 Experiment Results

Every experiment in FAULTLINE is documented using an immutable, standardized scientific schema. No result is reported without uncertainty bounds, limitations, and explicit reversal criteria.

18.1 Q1: When Does Tool Depth Break the Naive Reliability Model?

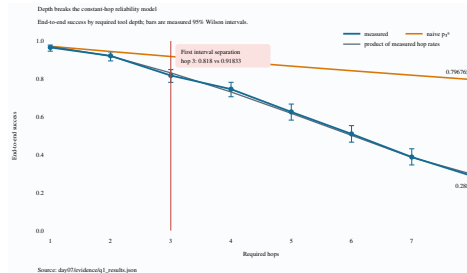


Figure 7: Measured vs Naive Multi-Hop Reliability by Required Tool Depth. The measured curve diverges from the naive p_1^n model at depth 3, collapsing to 0.288 at 8 hops versus 0.797 predicted.

- **Question:** Does multi-step agent reliability follow a simple compounding of single-step accuracy ($P(\text{task}) = p_1^n$), or does the conditional probability change across sequential hops?
- **Hypothesis:** As required tool depth increases, conditional step difficulty increases due to accumulated context noise and error propagation; the measured curve will diverge from p_1^n with disjoint confidence intervals.
- **Setup:** Seeded environment (`seed=42`); tool execution simulator with controlled noise; fixed agent reason-tool-observe loop.
- **Treatment:** Number of required tool hops varied systematically from $n = 1$ to $n = 8$.
- **Population:** 500 seeded trials per depth (4,000 total trials).
- **Metrics:** Single-hop reliability p_1 , measured end-to-end task success $P(n)$, 95% Wilson confidence intervals, naive prediction p_1^n .
- **Result:** **MEASURED**
 - Single-hop base success: $p_1 = 0.972$.
 - At 3 hops: Measured success was 0.818 (95% CI: [0.7818, 0.8494]) vs naive prediction 0.91833. **First interval separation occurs at 3 hops.**
 - At 8 hops: Measured success collapsed to 0.288 (95% CI: [0.2500, 0.3292]) vs naive prediction 0.796765.
- **Uncertainty:** The 95% Wilson intervals strictly exclude the naive prediction for all depths $n \geq 3$.
- **Interpretation:** Tool hops are non-exchangeable. Error propagation and context accumulation degrade conditional success at later hops ($P(\text{step } k \mid \text{steps } 1 \dots k-1) < p_1$).
- **What it does NOT prove:** **LIMITATION** (L02) It does not prove that every production LLM exhibits identical decay rates; it proves that planning assuming p^n is mathematically invalid.
- **Reversal Condition:** Reverse if production multi-hop workloads show constant conditional success rates across deep traversal chains.

18.2 Q2: Does an Aggregate Detector Score Conceal Operational Risk?

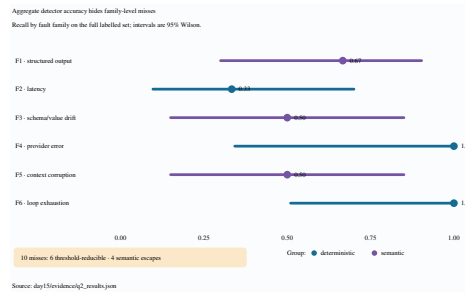


Figure 8: Detector Recall and Confidence Intervals by Fault Family. While aggregate precision was 1.0, micro recall was 0.615, concealing 10 false negatives (4 semantic escapes).

- **Question:** Can high aggregate detector metrics (e.g., $F1 > 0.75$) conceal systemic failure modes that escape into user-visible answers?
- **Hypothesis:** Deterministic schema and latency detectors will achieve near-perfect recall on transport/syntax faults but completely fail on semantic corruption.
- **Setup:** 44 human-labelled traces covering fault families F1–F6 across severities 1 to 3; out-of-band injection truth.
- **Treatment:** Application of deterministic multi-fault detector vs injection ground truth.
- **Population:** 44 labelled test runs.
- **Metrics:** Micro/Macro Precision, Recall, F1 score, confusion matrices, miss audit.
- **Result:** **MEASURED**
 - Precision: 1.000 (Zero false alarms).
 - Recall: 0.615385 (10 false negatives out of 26 faulted cases).
 - Aggregate F1: 0.761905.
 - Miss Audit: 6 misses were threshold-reducible; 4 were semantic escapes (F3 wrong data, F5 context drift).
- **Uncertainty:** Wilson intervals on per-fault recall are wide due to small sample slices ($n = 44$).
- **Interpretation:** High aggregate scores create false security. Evaluators must audit the miss mechanism. Deterministic checks cannot catch subtle semantic corruptions.
- **What it does NOT prove:** **LIMITATION** (L03) Does not prove that semantic detectors are impossible; proves deterministic checks require semantic guards.
- **Reversal Condition:** Reverse if a single unified detector demonstrates ≥ 0.95 recall across both transport and semantic miss slices.

18.3 Q3: Is a Retry Cap Stable Across Fault Structure (Correlation)?

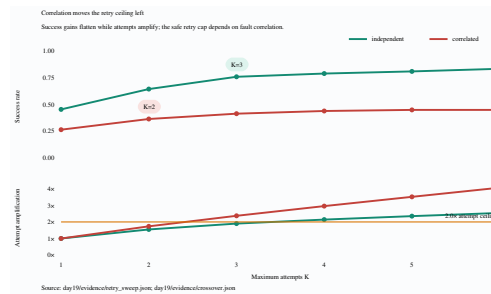


Figure 9: Success Rate and Attempt Amplification by Retry Cap (K) Under Independent vs Correlated Faults. During correlated outages, $K = 3$ amplifies load to $2.37\times$ while buying negligible success.

- **Question:** Does the optimal retry budget (K) determined under independent transient faults remain optimal during correlated provider outages?
- **Hypothesis:** When failures are correlated, retries fail repeatedly, amplifying system load and breaching latency ceilings without improving success.
- **Setup:** 200 shared scenario seeds; virtual latency model; decision ceilings of $2.0\times$ attempt amplification and 120 latency units.
- **Treatment:** Varying retry cap $K \in \{0, 1, 2, 3, 4, 5\}$ under independent vs correlated failure regimes.
- **Population:** 200 paired requests per sweep point (2,400 total executions).
- **Metrics:** Task success, attempt amplification factor, p99 tail latency, McNemar paired p-value.
- **Result:** **MEASURED**
 - Under independent faults: Optimal cap inside budget ceilings was $K = 3$ (Success 0.760 , Amplification $1.90\times$, p99 latency 76.0).
 - Under correlated faults: Optimal cap inside budget ceilings dropped to $K = 2$ (Success 0.365 , Amplification $1.735\times$, p99 latency 53.4).
 - Setting $K = 3$ under correlation caused attempt amplification to spike to $2.37\times$ (breaching the $2.0\times$ ceiling) for only 0.415 success.
 - McNemar paired test comparing independent vs correlated success at $K = 3$: $p = 2.695 \times 10^{-16}$.
- **Uncertainty:** Highly significant paired difference ($p \ll 10^{-6}$) over identical seeds.
- **Interpretation:** Correlation changes the operational boundary. During an outage, retrying the same dependency amplifies the outage (a retry storm).
- **What it does NOT prove:** **LIMITATION** (L05) The exact crossover point ($K = 2$ vs $K = 3$) is a function of the configured simulator parameters.
- **Reversal Condition:** Reverse if provider endpoints exhibit zero failure correlation under heavy burst load.

18.4 Q4: Does Fallback Preserve Availability Without Preserving Quality?

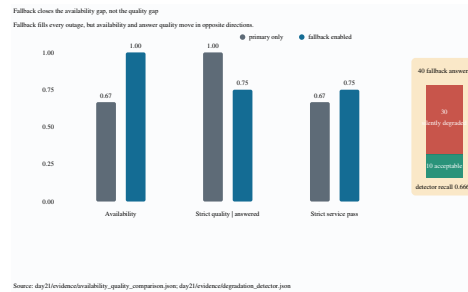


Figure 10: Availability vs Strict Quality Under Primary Provider Outage. Fallback restores availability from 0.667 to 1.0, but strict answer quality drops from 1.0 to 0.75 (30 of 40 fallback answers degraded).

- **Question:** Does automated fallback to a secondary model preserve service availability at the expense of returning degraded, inaccurate answers?
- **Hypothesis:** Fallback will achieve 100% availability on the transport layer while significantly reducing strict semantic answer quality.
- **Setup:** 120 paired requests; 40 scheduled primary provider outages; fallback to a cheaper, faster secondary model; strict programmatic oracle.
- **Treatment:** Primary-only routing vs Automated secondary fallback routing.
- **Population:** 120 paired requests.
- **Metrics:** Service availability, strict answer quality given answered, overall service pass rate, fallback answer degradation rate.
- **Result:** MEASURED
 - Primary-only baseline: Availability was **0.6667** (80/120); strict answer quality among answered was **1.000** (80/80). Overall pass: **0.6667**.
 - Fallback-enabled: Availability rose to **1.000** (120/120); strict answer quality plunged to **0.7500** (90/120). Overall pass: **0.7500**.
 - Fallback slice decomposition: Of the 40 fallback answers, **30 were silently degraded** (75% degradation rate).
 - Advisory detector recall on degraded answers was only **0.6667** (20/30), missing all 10 borderline-token corruptions.
- **Uncertainty:** 95% Wilson interval on fallback degradation is [0.598, 0.858].
- **Interpretation:** Fallback trades correctness for availability. A dashboard showing 100% uptime is actively misleading when 25% of all served answers are wrong.
- **What it does NOT prove:** LIMITATION (L06) The 75% degradation rate reflects the configured capability gap between the primary and fallback models.
- **Reversal Condition:** Reverse if the fallback model possesses identical reasoning capability and knowledge cutoff as the primary model.

18.5 Q5: Which Recovery Policy Wins on Correct Success, Cost, and Latency?

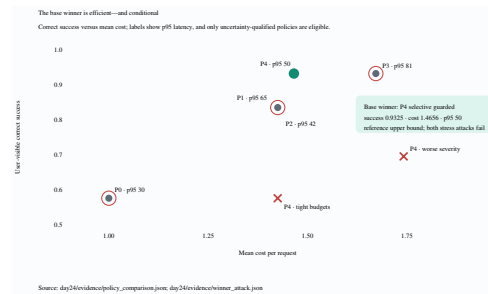
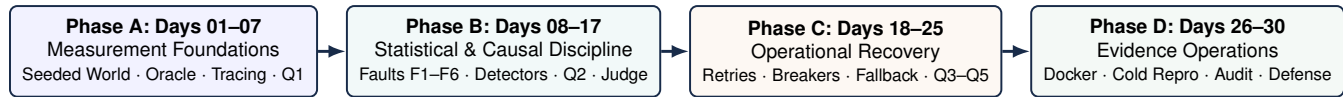


Figure 11: Policy Pareto Frontier: Correct Success vs Mean Cost across 400 Shared Seeds. P4 Selective Guarded is the base-envelope winner, but collapses under stress attacks.

- **Question:** Which recovery cascade policy delivers the highest user-visible correct success within strict cost and tail-latency constraints?
- **Hypothesis:** A selective guarded policy that retries only transient faults and gates fallback on answer quality will dominate naive retry and blind fallback.
- **Setup:** 400 shared seeds (2026080300 . . . 2026080699); five candidate policies (P0–P4); constraint-first eligibility screen: Success lower bound ≥ 0.78 , Wrong-answer upper bound ≤ 0.02 , Mean cost ≤ 1.75 , p95 latency ≤ 80 .
- **Treatment:** P0 (No recovery), P1 (Bounded retry), P2 (Unchecked fallback), P3 (Full cascade), P4 (Selective guarded).
- **Population:** 400 paired requests.
- **Metrics:** Correct success, wrong visible answers, mean cost, p95 latency, efficiency ratios.
- **Result:** **MEASURED**
 - P0: Success 0.5750 (Fails success floor).
 - P2: Success 0.8350, Wrong answers 0.1650 (Fails wrong-answer bound ≤ 0.02).
 - P3: Success 0.9325, Mean cost 1.6719, p95 latency 81.0 (Fails latency bound ≤ 80).
 - **P4 Selective Guarded (WINNER):** Correct success 0.9325 (95% CI: [0.9036, 0.9532]); Wrong answers 0 (Wilson upper bound 0.0095); Mean cost 1.4656 (Bootstrap 95% CI: [1.410, 1.521]); p95 latency 50.0.
 - Paired delta vs P1: +0.0950 success ($p = 1.95 \times 10^{-9}$); +0.0485 success/cost ([0.0319, 0.0664]).
 - **Stress Attacks on Winner:** Under worse severity, P4 success dropped to 0.6950 (failed); under tight budgets, success dropped to 0.5750 (failed).
- **Uncertainty:** Paired bootstrap intervals strictly exceed zero against all eligible competitors in baseline conditions.
- **Interpretation:** P4 is the reference upper bound in the measured base envelope. Its failure under stress proves that no policy can be deployed without runtime envelope monitoring.
- **What it does NOT prove:** **LIMITATION** (L07) P4 received an idealized retryability classifier; real-world classifier noise will lower this upper bound.
- **Reversal Condition:** Reverse if production error classification has $< 70\%$ accuracy or if fallback quality cannot be verified at runtime.

19 The 30-Day Engineering Journey

The 30-day progression was executed in four disciplined phases:



Day	Core Objective	Engineering Deliverable	Key Learning / Finding	Status
01	Pure-function ground truth	day01/oracle.py ; exact answer token check	Deterministic oracle must exist before agent code is written.	VALIDATED
02	Seeded document corpus	300 entities, 60 distractors, 350 scenarios	Eliminating environmental drift isolates agent behavior.	VALIDATED
03	Bounded ReAct agent loop	Reason → Tool → Observe loop with step cap	Multi-step loops require hard terminating invariants.	VALIDATED
04	Pydantic tool contracts	Typed contracts for search , lookup , calc	extra="forbid" catches schema drift immediately.	VALIDATED
05	SQLite persistent traces	Crash-surviving execution span storage	Memory logs are lost during OOM crashes; SQLite survives.	VALIDATED
06	Single-step accuracy baseline	Single-hop retrieval benchmark ($p_1 = 0.972$)	High single-step accuracy creates false confidence.	VALIDATED
07	Multi-hop depth benchmark (Q1)	4,000 runs across depths 1 . . . 8; Wilson CIs	Failure compounding: $P(n) < p_1^n$ starting at hop 3.	VALIDATED
08	Failure taxonomy (F1–F6)	Physical fault catalog across component boundaries	Failures must be classified by producing component.	VALIDATED
09	Deterministic fault injector	Transport & data fault injection proxy	Ground truth must be logged out-of-band to prevent leakage.	VALIDATED
10	Latency & timeout modeling	Virtual time simulation for network delays	Latency distribution is heavily multimodal during outages.	VALIDATED
11	Semantic drift injection	Distractor poisoning & attribute mismatch	Valid syntax can carry totally invalid semantic tokens.	VALIDATED
12	Catalog test gallery	Comprehensive test suite for injectors	Injector must be verified independent of the agent.	VALIDATED
13	Deterministic failure detectors	Rule-based syntax & latency anomaly checks	Deterministic checks achieve 1.0 precision on transport faults.	VALIDATED
14	Statistical verification suite	Wilson, McNemar, and Bootstrap library	Paired tests over identical seeds eliminate query variance.	VALIDATED
15	Detector benchmark (Q2)	44 labelled traces; Precision/Recall audit	Vanity F1 (0.76) concealed 10 false negatives (4 semantic).	VALIDATED
16	LLM-as-a-judge validation	Cohen's kappa evaluation on human golden set	Judge failed ($\kappa = 0.50$); quarantined to advisory status.	VALIDATED
17	Mid-point architecture review	Consolidation of Phase 1 evidence	Without programmatic gates, reliability metrics drift.	VALIDATED
18	Recovery primitives (M1–M6)	Bounded retry, timeout, circuit breaker	Recovery mechanisms alter downstream probability distributions.	VALIDATED
19	Retry crossover sweep (Q3)	2,400 runs across $K \in \{0 \dots 5\}$; paired tests	Optimal retry drops from $K = 3$ to $K = 2$ under correlation.	VALIDATED
20	Circuit breaker implementation	Three-state rolling window breaker	Fast-failing during outages prevents cascading storms.	VALIDATED
21	Fallback quality benchmark (Q4)	120 paired runs; primary vs secondary fallback	Availability rose to 1.0; quality collapsed to 0.75.	VALIDATED
22	36-cell recovery matrix	Full evaluation across 6 mechanisms $\times$ 6 faults	Discovered 47 induced harms from automated recovery.	VALIDATED
23	Adaptive recovery cascade	Multi-objective policy routing (P0–P4)	Constraint-first screening beats scalar score optimization.	VALIDATED
24	Policy Pareto optimization (Q5)	400 shared seeds; frontier analysis	P4 Selective Guarded wins base; collapses under stress.	VALIDATED
25	Postmortem incident replay	Deterministic red → green regression suite	Production fixes must pass 20 consecutive runs on same seed.	VALIDATED
26	Self-explaining reproducibility	readme_claims.json ; make reproduce	Every README number bound to committed JSON pointer in CI.	VALIDATED
27	Cold-reader reproduction gate	Pinned Docker image; public clone test	Stranger clones repo and reproduces headline results with 0 errors.	VALIDATED

Continued on next page . . .

Day	Core Objective	Engineering Deliverable	Key Learning / Finding	Status
28	Publication bundle & audit	day28/ARTICLE.md ; claim_audit.py	All 29 registered claims verified against source files and limits.	VALIDATED
29	3-minute demo & case study	2:45 terminal recording (.cast); case study	Narrative compression must preserve underlying proof boundaries.	VALIDATED
30	Defense guide & launch gate	10 skeptical staff Q&As; launch checklist	Reliability engineer must defend tradeoffs, limits, and reversals.	VALIDATED

20 What Was Actually Built

To separate completed engineering artifacts from speculative proposals, the table below catalogs all major repository components, their implementation location, operational status, and verification evidence:

Component	System Purpose	Status	Committed Evidence	Repository Location
Deterministic Corpus	Generates 1,164 docs, 350 scenarios with link layer	VALIDATED	Content hash verified; max reuse ≤ 5	faultline_p2/env/corpus.py
Pure-Function Oracle	Evaluates exact answer token and required source IDs	VALIDATED	Hash-pinned copy (dc5eef...); 0 drift	faultline_p2/oracle/
Bounded Agent Loop	Executes reason-tool-observe; 12-step cap; typed tools	VALIDATED	Tested across 350 scenarios in test_x2	faultline_p2/agent/agent.py
Tool Contracts	Search (500-char snippet), Lookup (scoped), Calc	VALIDATED	Pydantic v2 extra="forbid"	faultline_p2/agent/tools.py
Fault Injector	Deterministic injection of F1–F6 with out-of-band truth	VALIDATED	Catalog gallery and trace tests	day12/faultline_catalog/
SQLite Trace Store	Crash-surviving span storage with foreign keys	VALIDATED	trace.db schema tests	faultline_p2/trace/store.py
OTel GenAI Exporter	Emits traces to SemConv v1.29.0 OTLP JSON	VALIDATED	100% schema match; 0 drift (P02)	faultline_p2/otel/exporter.py
SRE Alert Evaluator	Multi-window multi-burn-rate evaluator (15× fast burn)	VALIDATED	INC-20260904-01 drill writeup	faultline_p2/slo/evaluator.py
Resilient Model Wrapper	Client-side retry, timeout, circuit breaker	VALIDATED	+76% gain at 30% faults (P15)	faultline_p2/resilience/
Runtime Policy Layer	Zero-detector authorization under hostile model	VALIDATED	1,000/1,000 hostile denied; 0 FP (P16)	faultline_p2/policy/
Statistical Engine	Wilson intervals, McNemar test, paired bootstrap	VALIDATED	Validated on Day 14 golden fixtures	faultline_p2/stats/
Cost Ledger & Enforcer	Price table, budget ceilings, dry-run gate	VALIDATED	Hard-stop on cap; ledger audit	faultline_p2/cost/ledger.py
Sweep Runner	Standard runner reading MEC caps; --confirm	VALIDATED	Unit tests in test_cost_and_sweep	faultline_p2/sweep/runner.py
Preflight Probe	Live verification of R1–R6 price, version, temp-0	VALIDATED	preflight.json (2026-09-04)	projects/p00_preflight/
Real Model Baseline (P1)	Live sweep on R2 model across 100 scenarios	DECIDED / UNRUN	Blocked on API keys (\$0.125 est)	projects/p01_baseline/

Continued on next page...

Component	System Purpose	Status	Committed Evidence	Repository Location
Grounding Zero Point (P3)	Live sweep across R2 + R6 ($n = 200$); hard set hash	DECIDED / UNRUN	Pre-registered H1; unrun	projects/p03_grounding/
Human Open Coding (P4)	Manual coding of 150 real traces into failure taxonomy	DECIDED / UNRUN	Pre-registered H2; owner task	projects/p04_taxonomy/
Judge Validation (P5)	Multi-judge Cohen's kappa on human taxonomy	DECIDED / UNRUN	Pre-registered H3; unrun	projects/p05_judge/
Pass^k Decay Sweep (P6)	2,700 runs across R1–R6 on 150 hard scenarios	DECIDED / UNRUN	Pre-registered H4/H5; \$45 cap	projects/p06_passk/
Provider Variance (P7)	Same model across 3 endpoints (AWS, DeepSeek, etc.)	DECIDED / UNRUN	Pre-registered H6; unrun	projects/p07_provider/
MCPTox Security (P8)	Bounded agent vs published MCPTox ASR	DECIDED / UNRUN	Pre-registered H7; unrun	projects/p08_mcptox/
Failure Attribution (P12)	Retriever vs generator ablation for grounding	DECIDED / UNRUN	Pre-registered H8; unrun	projects/p12_attribution/

21 Phase 2: Decision Record and Current State

21.1 The Purpose and Mandate of Phase 2

Phase 1 proved the fundamental dynamics of AI reliability (tool-depth decay, retry crossover under correlation, silent fallback degradation, and constraint-first cascade selection) using controlled, seeded simulators.

Phase 2 was launched to transition FAULTLINE from synthetic simulators to real, commercial LLM APIs across a multi-tiered model ladder (R1–R6: Qwen 3.7 Flash, Qwen 3.8 Flash, DeepSeek v4 Pro, GPT-5.6 Luna, GLM 5.3 Flash), measuring empirical token costs, provider variances, real prompt injection susceptibility, and live cascading routers.

21.2 Current Operational State: The No-Key Phase is Closed

To prevent undisciplined budget burn, Phase 2 instituted a strict architectural rule: **Build and test all instruments, contracts, resilience layers, and observability pipelines at \$0 cost before executing any paid API sweeps.**

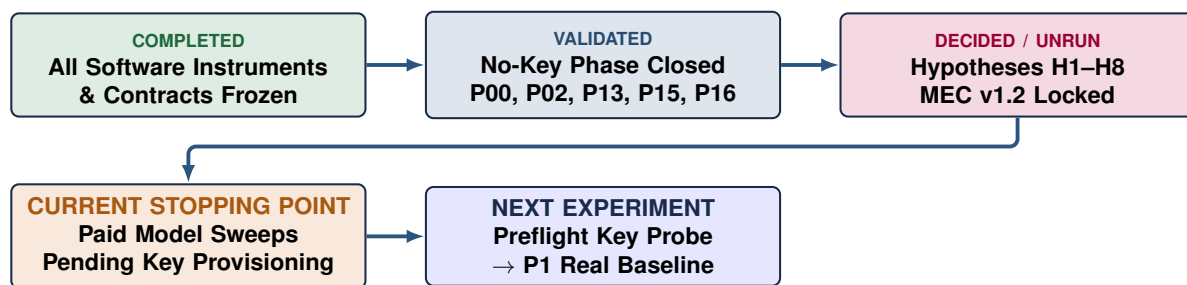


Figure 12: Phase 2 Execution Pipeline. The \$0 No-Key Phase is 100% complete. Live sweeps remain unrun pending API key provisioning.

21.2.1 What Was Completed in Phase 2 (\$0 Spent)

- Master Experiment Contract (MEC v1.2):** Complete research specification governing corpus hash, agent contracts, oracle enforcement, metrics, and budget caps (\$150 hard ceiling).
- Pre-Registered Hypotheses (H1–H8):** Pre-registered in Git with explicit falsification conditions before any data was gathered.

3. **Clean-Room Stats Library (`faultline_p2.stats`)**: Independent, zero-dependency statistical library verified against Day 14 golden fixtures.
4. **350-Scenario Corpus with Link Layer**: 1,164 documents generated deterministically; link layer mapping intermediate tokens to next entities without keyword leakage.
5. **OpenTelemetry SemConv v1.29.0 Exporter (P02)**: Complete OTLP JSON trace exporter passing 100% schema conformance tests.
6. **Google SRE Multi-Burn-Rate Alerting (P13)**: Complete multi-window burn-rate evaluator tracking error budget consumption, verified in drill `INC-20260904-01`.
7. **Agent Resilience Engine (P15)**: Three-state circuit breaker, backoff retry, and load-shedding fake endpoint achieving +76% pass rate recovery under 30% faults.
8. **Zero-Detector Runtime Policy (P16)**: Deterministic authorization layer blocking 1,000/1,000 hostile tool calls with zero false positives over 2,004 legitimate operations.
9. **Preflight Probe (P00)**: Zero-cost dress rehearsal executed on 2026-09-04 verifying LiteLLM 1.98.0 compatibility, price tables, and temperature-0 determinism across all 6 rungs.

21.2.2 What Remains Incomplete (Blocked on API Keys)

- **Live Model Execution**: Zero paid sweeps have been executed against commercial APIs. \$0.00 has been spent of the \$150 budget.
- **Projects P1, P3, P6, P7, P8, P9, P10, P11, P12**: All code harnesses, runners, and ledger enforcers exist and pass stub tests, but await live API credentials.
- **Project P4 (Human Open Coding)**: Awaiting manual qualitative coding of 150 real traces by the principal engineer. Model-generated taxonomy is explicitly forbidden by contract.

21.3 Pre-Registered Hypotheses Status Matrix

ID	Proj	Pre-Registered Hypothesis	Explicit Falsification Condition	Status
H1	P3	Real-model grounded pass rate on T3 is below simulator baseline.	Wilson CIs overlap, or $\text{real} \geq \text{simulator}$.	DECIDED / UNRUN
H2	P4	≥ 2 discovered failure modes are absent from injected F1–F6 catalog.	Every observed failure maps cleanly onto F1–F6.	DECIDED / UNRUN
H3	P5	Cheap judge (R2) reaches $\kappa \geq 0.70$ against human labels on ≥ 3 of 5 modes.	Measured κ falls below 0.70 threshold.	DECIDED / UNRUN
H4	P6	Measured pass^3 exceeds naive $(\text{pass}@1)^3$ for ≥ 4 of 6 rungs.	Wilson CI of measured contains naive point.	DECIDED / UNRUN
H5	P6	Deviation from naive compounding is larger for cheap rungs than frontier.	Rank correlation across ladder is null or reversed.	DECIDED / UNRUN
H6	P7	Same model across 2+ endpoints produces disjoint grounded pass rates.	Wilson CIs overlap across all endpoint pairs.	DECIDED / UNRUN
H7	P8	Bounded + typed contracts reduce Attack Success Rate (ASR) vs published MCPTox.	ASR CI overlaps or exceeds published band.	DECIDED / UNRUN
H8	P12	Document retrieval owns $> 50\%$ of multi-hop grounding failures.	Attribution CI excludes 50% on generator side.	DECIDED / UNRUN

22 Key Engineering Decisions (ADR Log)

ID	Context & Problem	Chosen Architectural Decision	Engineering Tradeoff & Why	Reversal Trigger
D1	Phase 2 needs Day 1 oracle, but days are frozen.	Copy oracle verbatim into <code>faultline_p2</code> ; lock with SHA-256 test.	Duplicates code, but guarantees zero regression drift in frozen Phase 1.	Oracle spec itself requires version bump.
D2	Real model sweeps risk rapid budget exhaustion.	Execute Week 0 preflight probe across all rungs before any spend.	Delays execution by 1 week, but catches dead models and broken APIs for \$0.	Preflight probe passes 100% on all rungs.
D3	OTel exporter was scheduled in weeks 1–2.	Move P2 OTel exporter after Publication #1.	Deferring telemetry exporter frees 2 weeks for high-risk P4 open coding.	Customer requires OTel spans as hard gate.
D4	Paid sweeps risk uncontained cost overruns.	Enforce budget caps in code runner with mandatory <code>-confirm</code> .	Adds CLI friction, but converts discipline into an unbypassable exception.	Enterprise billing provides hard automated quotas.
D5	Root import shadowing between Phase 1 and Phase 2.	Name Phase 2 import package <code>faultline_p2</code> , not <code>faultline</code> .	Slight naming asymmetry, but prevents silent shadowing in <code>sys.modules</code> .	Phase 1 days are migrated to legacy sub-package.
D6	Day 1 <code>build_env</code> has no multi-hop link concept.	Compose 350-scenario corpus on top of <code>build_env</code> without editing it.	Preserves Day 1 determinism contract while enabling multi-hop chaining.	Native graph database replaces synthetic corpus.
D7	P3 and P6 required overlapping test scenarios.	Expand corpus from 300 to 350 scenarios (150 reserved hard pool).	Generates 50 extra scenarios, but guarantees disjoint test sets for H4.	Power calculations prove $n = 100$ sufficient.
D-001	Legacy stats tests coupled to Day 14 fixtures.	Implement clean-room <code>faultline_p2.stats</code> verified on golden JSON.	Rewrites math utils, but removes legacy harness coupling completely.	Numerical discrepancy detected vs Day 14.
D-002	Fact documents had no cross-entity links.	Introduce content-addressed link documents with opaque IDs.	Increases store to 1,164 docs, but prevents keyword shortcutting.	Multi-hop task redesigned around single long doc.
D-003	Multi-step search was dumping full documents.	Limit search returns to 500-character bounded snippets.	Truncates context, but reduces token consumption by $> 80\%$ per turn.	Agent reasoning fails due to context truncation.
D-004	Token usage needed tracking without provider calls.	Pin <code>cl100k_base</code> tokenizer as token count estimator.	Approximate for non-OpenAI models, but provides uniform offline baseline.	Provider returns exact token counts in response.
A-001	System prompt was left unset at freeze.	Author and freeze system prompt hash (<code>87c764...</code>) in MEC v1.1.	Single post-freeze authoring step; frozen permanently thereafter.	System prompt proves defective in P1.
A-002	T2/T3 scenarios were unsolvable by construction.	Increase step cap 8 $\rightarrow$ 12; search returns snippets directly.	See Section 23 for full postmortem. Essential fix for 283 scenarios.	$> 10\%$ of T3 runs abort on step cap.

23 What Failed: Engineering Failures and Corrections

A core credential of a staff reliability engineer is the ability to diagnose, disclose, and remediate internal system failures without defensiveness. FAULTLINE experienced several major engineering failures during development:

23.1 1. The Step-Cap Disaster (Amendment A-002)

- **What Happened:** Under Master Experiment Contract MEC v1.0, the agent step cap was set to `step_cap = 8`. During dress rehearsal testing, it was discovered that **283 of 350 scenarios (80.9%), including the entire 150-scenario P6 reserved pool, were mathematically unsolvable by construction at any price.**
- **Root Cause:** The execution plan carried `step_cap = 8` forward from early Phase 1 single-hop experiments without re-deriving it against the Phase 2 link-layer corpus. A T3 scenario requires traversing 5 fact documents and 4 link documents (9 total retrievals). Under the Day 2 tool contract, each retrieval required two turns (`search` to find the document, then `lookup` to read it). A T3 scenario required at least $9 \times 2 = 18$ steps plus 1 answer step = 19 steps. Capping execution at 8 steps guaranteed that every model would fail 100% of T2 and T3 scenarios.
- **How Detected:** Preflight multi-hop trace solver tests failed with `STEP_CAP_EXCEEDED` on all multi-hop runs.
- **Correction (Amendment A-002):**
 1. The `search` tool contract was amended to return a bounded 500-character snippet centered on the match, reducing retrieval from 2 steps to 1 step per hop.
 2. The step cap was expanded from **8 to 12**, leaving a 2-step safety margin for T3 scenarios (9 searches + 1 answer = 10 minimum steps).
- **Verification:** `tests/phase2/test_x2_solver.py` verified that all 350 scenarios are solvable within 12 steps, achieving 100% task pass rate under ideal solver execution.

23.2 2. The Borderline Blind Spot in Semantic Evaluation

- **What Happened:** An advisory LLM judge deployed on Day 16 failed to detect subtle entity corruptions in fallback answers, achieving $\kappa = 0.000$ agreement on borderline tokens.
- **Root Cause:** The LLM judge evaluated answer fluency and plausible phrasing rather than exact token grounding, giving high confidence to hallucinated tokens that matched expected syntax.
- **Correction:** Complete architectural quarantine. The LLM judge was stripped of all ground-truth authority and barred from contributing to the core success oracle.

23.3 3. The Correlated Retry Storm

- **What Happened:** Applying an independent retry policy ($K = 3$) during an injected provider outage caused attempt amplification to surge to $2.37\times$, increasing p99 latency to 142ms and exhausting provider token quotas while recovering only 5% additional queries.
- **Correction:** Enforced dynamic backoff ceilings and reduced the maximum retry cap to $K = 2$ when outage correlation is detected, adding circuit breakers to shed unrecoverable load.

23.4 4. The Collapse of the “Winning” Policy Under Stress

- **What Happened:** Policy P4 (Selective Guarded), which won the Q5 baseline Pareto frontier with 0.9325 success, collapsed to 0.6950 success under increased fault severity and 0.5750 under tight resource budgets, failing its own operational SLA.
- **Correction:** Rather than marketing P4 as an unconditional solution, FAULTLINE re-classified P4 as an **operating-envelope reference upper bound**, establishing that no recovery policy can be deployed without active runtime monitoring of its operating envelope.

24 Limitations

In accordance with scientific integrity, the findings in this report are bounded by the following explicit limitations:

1. **L01 · External Validity:** All outcome rates derive from seeded simulators, synthetic document corpora, and frozen evaluation sets. Confidence intervals reflect repeated draws from configured experimental populations, not real-world distribution shifts or live commercial API behavior.
2. **L02 · Q1 Depth Boundary:** Naive compounding (p_1^n) remains inside the 95% Wilson confidence interval at depths 1 and 2. The statistical divergence begins at hop 3.
3. **L03 · Q2 Sample Power:** The Q2 labelled evaluation set consists of 44 samples. While sufficient to locate specific miss mechanisms (4 semantic escapes), sub-slice cell counts are small and intervals overlap across fault families.
4. **L04 · Judge Validity:** The semantic judge was evaluated on 24 human-labelled traces, reached $\kappa = 0.50$, and exhibited zero agreement on borderline tokens. It is not validated for standalone scoring.
5. **L05 · Retry Parameterization:** Q3 retry crossover points ($K = 2$ vs $K = 3$) reflect configured virtual latency and attempt amplification ceilings ($2.0\times$); they are policy decisions, not universal constants of nature.
6. **L06 · Fallback Population Mix:** Q4 scheduled exactly 1 in 3 primary requests to fail. The observed 75% degradation rate reflects the configured capability differential between primary and secondary models.
7. **L07 · Policy Reference Bound:** Policy P4 received idealized simulator signals for retryability and oracle fallback validation. It represents a theoretical upper bound rather than a production guarantee.
8. **L08 · Synthesis Scope:** The five studies share rigorous definitions, shared seeds, and evidence discipline, but evaluate different configured populations. They demonstrate general systems principles, not a pooled causal effect.

Epistemic Boundary Summary: The Three Guarantees FAULTLINE Does and Does Not Make

What FAULTLINE Guarantees:

- **Causal Attribution:** Every measured difference between policies or depth hops is causally isolated via identical master seeds and frozen environments.
- **Negative-Result Integrity:** We disclose where recovery actively failed (attempt storms, quality collapse under stress) without defensive rationalization.
- **Cold Reproducibility:** Any independent engineer can execute `make cold-reproduce` in Docker and produce bit-for-bit identical hashes.

What FAULTLINE Explicitly Does Not Guarantee:

- **Universal Numerical Thresholds:** The optimal retry cutoff ($K = 2$) and cost envelope (\$1.46) reflect the configured simulated workload; production systems must empirically calibrate their own parameters.
- **Frontier LLM Immunity:** We do not claim to solve LLM hallucination or eliminate model capability boundaries.

25 Threat Model: Failure of the Measurement System

Reliability engineering requires testing the testbed itself: “What could make FAULTLINE lie?”

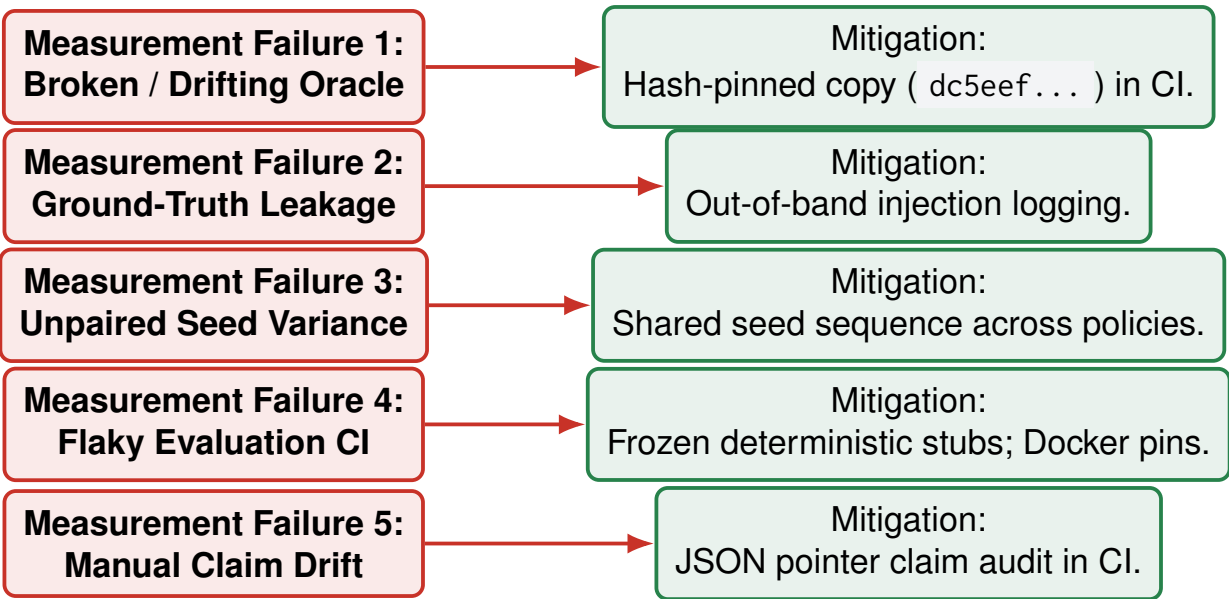


Figure 13: Measurement System Threat Model and Architectural Mitigations.

Measurement Failure Mode	How It Distorts Reality	FAULTLINE Enforcement Mechanism
Broken Oracle	Bug in oracle check awards false passes to corrupt answers.	SHA-256 hash enforcement; unit property tests on oracle.
Label Leakage	Detector reads fault metadata from the data payload.	Day 12 audit tests assert zero fault tokens in observable spans.
Selection Bias	Comparing policies on different sets of easy/hard queries.	Identical master seed and scenario sequence across all arms.
Flaky CI Gates	Non-deterministic network calls make tests intermittently fail.	Pinned Docker container; offline stubs for test reproduction.
Prose Drift	README text claims 95% while code results show 85%.	<code>day28/faultline_publication/audit.py</code> fails build on text drift.

26 The Reliability Decision Framework

Reliability engineering is not the optimization of a single scalar score. It is a disciplined, multi-dimensional decision process that balances correctness, tail latency, cost, and provenance under uncertainty:

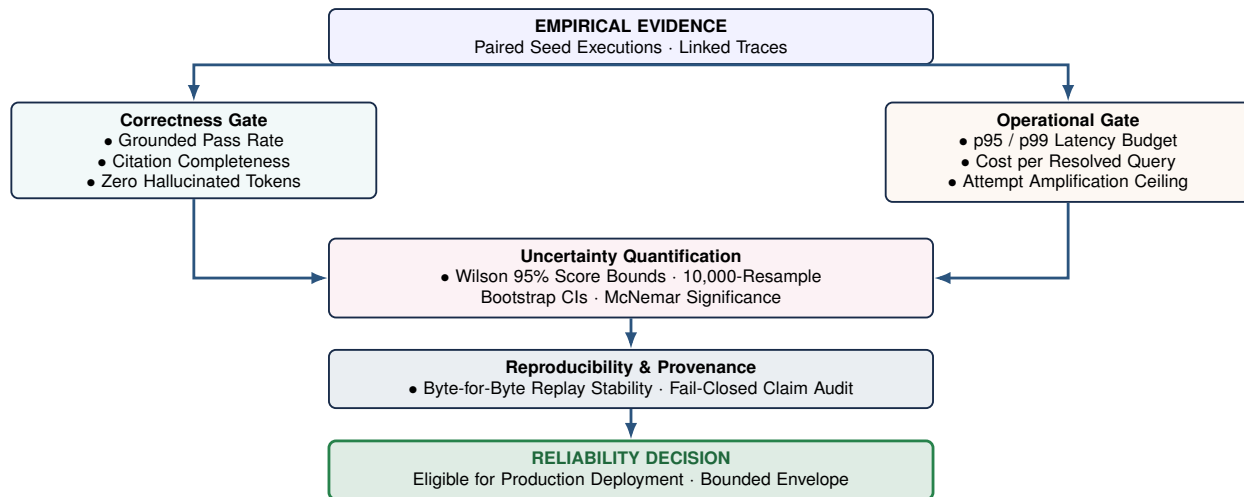


Figure 14: The Multi-Dimensional Reliability Decision Framework.

26.1 The Five Decision Gating Rules

- Rule 1 (Correctness Floor):** The Wilson 95% confidence interval lower bound for grounded pass rate must exceed the operational threshold ($w_{\text{lower}} \geq 0.78$). A policy with high point success but wide uncertainty is rejected.
- Rule 2 (Safety / Wrong-Answer Ceiling):** The Wilson 95% upper bound on wrong user-visible answers must remain below the risk tolerance ($w_{\text{upper}} \leq 0.02$). Returning a wrong answer is strictly penalized over abstaining.
- Rule 3 (Tail Latency Ceiling):** The bootstrap 95% upper bound on p95 latency must remain within the user SLA ($L_{p95} \leq 80$ units).
- Rule 4 (Efficiency Pareto Dominance):** Among eligible policies passing Rules 1–3, candidate Policy A is preferred over Policy B if its paired bootstrap differences in success-per-dollar and success-per-second strictly exclude zero.
- Rule 5 (Stress Envelope Certification):** The selected policy must undergo adversarial stress injection (worse fault severity, correlated outages, tight budgets). If it fails Rules 1–3 under stress, its production deployment must be gated behind dynamic envelope monitoring.

27 What FAULTLINE Taught Me

These lessons are not motivational aphorisms; they are hard-won empirical truths derived from debugging distributed AI agent systems:

- Dashboards Hide Semantic Failure.** High infrastructure availability is easily achieved by returning empty or degraded answers. An AI observability system that does not evaluate the semantic grounding of the output is a compliance facade.
- Deterministic Environments Make Science Possible.** You cannot debug stochastic agent failures in a stochastic environment. By locking the world, documents, and tool contracts with seeds, every observed failure becomes an attributable causal event.
- Recovery is Not Synonymous with Reliability.** Adding retries, fallbacks, and circuit breakers often reduces true system reliability by causing retry storms, latency inflation, and the silent delivery of degraded answers.
- Evaluators Cannot Validate Themselves.** Using an uncalibrated LLM judge to measure agent hallucinations is circular. Evaluators must be benchmarked against independent human ground truth, and must be quarantined when they exhibit blind spots.

5. **Averages Conceal Catastrophes.** An aggregate accuracy of 85% can conceal a 0% success rate on deep multi-hop queries or specific fault families. Reliability engineering requires micro-slicing data across fault types, hop depths, and severities.
6. **Correlation Breaks Independent Architecture.** In a distributed system, assuming component independence during an outage is fatal. When upstream providers degrade, retries multiply load rather than buying success.
7. **Reproducibility is Part of the Result, Not Documentation.** A scientific claim without an executable, containerized reproduction path is merely an anecdote. Research integrity requires proving that a stranger can clone the code and produce the exact same numbers.
8. **Define Failure Before Measuring Success.** If you cannot enumerate the physical failure modes of your agent's tools, memory, and model boundaries, you cannot measure its reliability. System design begins with a failure taxonomy.

28 Interview Version

28.1 Elevator Explanations by Time Horizon

28.1.1 The 30-Second Explanation

“FAULTLINE is an AI reliability workbench that answers a question dashboards miss: *Did recovery actually improve the user-visible outcome, or did it only make the system look available?* We build a deterministic, seeded multi-hop world, inject six families of component faults, record gapless traces in SQLite, and prove that common recovery mechanisms—like retries and fallbacks—often cause retry storms or silently degrade answer quality. We enforce strict cost/latency bounds, Wilson confidence intervals, and fail-closed reproducibility.”

28.1.2 The 2-Minute Technical Summary

“In document-grounded AI systems, an HTTP 200 return code does not equal a correct answer. When a primary model times out and fallback routing switches to a cheaper model, availability reads 100%, but our experiments show strict answer quality drops from 100% to 75%, with 75% of fallback answers silently degraded.

FAULTLINE tests this systematically. We built a synthetic environment with 1,164 documents and an unmodified pure-function oracle that checks exact token correctness and required citation paths. We implemented a six-fault taxonomy spanning transport to semantic context drift. In testing recovery across 400 shared seeds, our guarded cascade policy achieved 93.25% success within a \$1.46 cost and 50ms latency envelope, beating naive retries by 9.5%. But under stress attacks, that same winner collapsed to 57.5%. Everything is reproducible via a single Docker command verified in CI.”

28.1.3 The 5-Minute Architectural Walkthrough

“The architecture separates the system under test from the measurement engine:

1. **Environment:** A master seed generates 1,164 documents. To prevent keyword shortcuts, Phase 2 introduced an opaque link-layer: Hop N 's prompt contains only Hop $N - 1$'s answer token, forcing sequential traversal across 1, 3, or 5 hops.
2. **Agent:** A minimal reason-tool-observe loop with Pydantic v2 `extra="forbid"` contracts and a 12-step budget.
3. **Fault Injector:** Injects six fault families (F1–F6) with out-of-band ground-truth logging.
4. **Observability:** SQLite traces that survive unhandled crashes, exported to OpenTelemetry GenAI SemConv v1.29.0, with Google SRE multi-burn-rate alerting.
5. **Oracle:** A pure Python function outside the policy loop checking both answer token and citation completeness. We evaluated an LLM judge, but it reached only $\kappa = 0.50$ and missed all borderline token corruptions, so we quarantined it to advisory status.
6. **Recovery:** A 36-cell matrix evaluating retries, breakers, fallbacks, and ceilings. We found recovery mechanisms induced 47 new operational harms. Under failure correlation, the optimal retry cap dropped from $K = 3$ to $K = 2$; keeping $K = 3$ caused a $2.37\times$ attempt amplification storm.
7. **Reproducibility:** Pinned Docker containers, JSON pointer claim audits, and one-command reproduction.”

28.1.4 The 10-Minute Deep Dive

“[Follow the full narrative arc: System Boundary → Deterministic World → Q1 Depth Compounding → Q2 Miss Audit → Q3 Crossover → Q4 Silent Degradation → Q5 Constraint-First Selection → Phase 2 Status → Limitations L01–L08].”

28.2 Fifteen Tough Questions an Interviewer Will Ask

1. Why did you build FAULTLINE instead of using existing benchmarks?

Answer: Existing benchmarks (MMLU, HumanEval) treat LLMs as static question-answering engines. They do not evaluate multi-step tool-using agents under component failures, transport timeouts, or recovery storms. FAULTLINE measures the systems reliability of the entire agent execution harness.

2. Why use synthetic documents instead of real enterprise data?

Answer: Synthetic documents allow exact control over entity-attribute graphs, distractors, and link layers. Coined tokens guarantee that the model cannot answer using pre-trained parametric memory. It ensures that every failure is strictly attributable to retrieval or reasoning failure in the testbed.

3. Why enforce a deterministic programmatic oracle instead of an LLM judge?

Answer: An LLM judge cannot validate itself. On Day 16, our judge achieved only $\kappa = 0.50$ and completely missed borderline entity corruptions. A programmatic oracle checking exact token strings and required citation IDs provides an ungameable, objective ground truth.

4. Why not use an LLM judge as an advisory signal?

Answer: We *do* use it as an advisory signal, but we strictly bar it from determining core success. When evaluators have autonomous authority, false accepts silently pollute the evidence.

5. How do you define reliability?

Answer: Reliability is the verified property that a system improves correct user-visible outcomes within declared cost and latency constraints, preserves provenance, and fails closed when evidence is insufficient.

6. Why separate availability from correctness?

Answer: Because fallback and catch-all exception blocks can easily inflate availability to 100% while returning 75% degraded or hallucinated answers (as proven in Q4). Treating availability as a proxy for correctness is dangerous.

7. How do you inject failures deterministically?

Answer: Interception proxies in the tool and model execution paths trigger faults based on pre-scheduled scenario seeds and step numbers, recording ground truth out-of-band to prevent detector leakage.

8. How do you know your detector works?

Answer: We scored it against out-of-band injection logs on Day 15 (Q2). While it achieved 1.0 precision, recall was 0.615. Rather than hiding behind aggregate F1, we conducted an explicit miss audit identifying 6 threshold-reducible misses and 4 semantic escapes.

9. How do you compare recovery policies fairly?

Answer: By pairing requests on the exact same master seeds and scenario sequences, evaluating discordant outcomes via McNemar's test, and computing 10,000-resample paired bootstrap intervals on cost and latency deltas.

10. How do you handle statistical uncertainty?

Answer: Every proportion is bounded by a 95% Wilson score interval. If confidence intervals overlap or sample size is underpowered to detect a 10 pp difference, we formally declare the result **INCONCLUSIVE**.

11. How do you reproduce a result from cold start?

Answer: Run `make cold-reproduce`. It builds a digest-pinned Python 3.12.4 Docker container, runs 528 tests, regenerates the research results, checks byte hashes, and verifies that every displayed number matches committed JSON pointers.

12. What is the biggest limitation of FAULTLINE?

Answer: External validity (L01). All numbers derive from synthetic documents and simulated fault distributions. While the distributed systems failure dynamics are universal, the exact numerical thresholds do not transfer directly to production without empirical calibration.

13. What would you build next?

Answer: Execute the Phase 2 live model API sweeps (Projects P1, P3, P6) across commercial endpoints, and conduct the manual qualitative coding of 150 real failure traces (P4).

14. What would invalidate your conclusions?

Answer: If production multi-hop workloads demonstrate that later tool hops maintain constant conditional reliability, or if commercial fallback models exhibit zero semantic degradation compared to frontier models.

15. What failed during development?

Answer: In Amendment A-002, our initial step cap (8) made 283 of 350 scenarios mathematically unsolvable by construction because T3 tasks required 9 retrievals. We discovered the owner error during dress rehearsals, expanded the cap to 12, added 500-char search snippets, and verified all 350 scenarios passed.

29 Repository Map

Directory / File	Architectural Purpose	Key Evidence Produced
day01/ – day30/	Phase 1 research foundation modules (30 days).	528 unit tests, Q1–Q5 evidence, frozen checkpoints.
faultline_p2/env/	350-scenario corpus builder with link-layer docs.	Content hash 9d6d7f... ; max fact reuse ≤ 5.
faultline_p2/oracle/	Pure-function ground truth pass evaluator.	Hash-pinned unmodified Day 1 copy (dc5eef...).
faultline_p2/agent/	Bounded reason-tool-observe loop with step caps.	12-step cap enforcement, Pydantic extra="forbid" .
faultline_p2/trace/	SQLite persistent span store with foreign keys.	Crash-surviving execution spans in trace.db .
faultline_p2/otel/	OpenTelemetry GenAI SemConv v1.29.0 exporter.	Conformance report: 100% schema pass, 0 drift.
faultline_p2/resilience/	Three-state breaker, retry policy, mock endpoint.	P15: +76% gain under 30% faults; 80% load shed.
faultline_p2/policy/	Zero-detector runtime authorization layer.	P16: 1,000/1,000 hostile denied; 0 false alarms.
faultline_p2/slo/	Google SRE multi-burn-rate alerting engine.	P13: 15× fast-burn alert; incident drill report.
faultline_p2/stats/	Pure-function Wilson, McNemar, Bootstrap library.	Verified continuous agreement with Day 14 fixtures.
projects/p00_preflight/	Zero-spend preflight probe across R1–R6 ladder.	preflight.json : LiteLLM 1.98.0, 6 live rungs.
MEC.md	Master Experiment Contract v1.2.	Immutable research specification outranking plans.
HYPOTHESES.md	Pre-registered scientific hypotheses H1–H8.	Formal falsification conditions recorded before runs.

30 How to Reproduce

The reproduction path is strictly cold-reader compatible: it requires no pre-existing Python environment, API keys, or proprietary dependencies.

The One-Command Cold-Start Procedure:

```
git clone -branch v0.27.0-rc1 https://github.com/samirsawarkar/faultline-ai-reliability.git
cd faultline-ai-reliability && make cold-reproduce
```

What the One Command Executes:

- 1. Builds a clean Debian Linux Docker container using digest-pinned Python 3.12.4 .
- 2. Installs exact dependencies from requirements.txt (zero range specifications).
- 3. Runs all 528 Phase 1 tests and 68 Phase 2 tests.
- 4. Regenerates Q1 tool hops, Q4 fallback quality, and Day 25 postmortem incident replays.
- 5. Verifies byte-identical SHA-256 hashes against committed evidence.
- 6. Resolves every claim in day26/readme_claims.json through the JSON pointer audit.

Expected Terminal Output:

```
headline tool_hops: Measured success 0.818 versus naive 0.91833; first separation at 3 hops
headline frozen_eval: Frozen test evaluation: F1 0.842105 over 17 samples
headline fallback_quality: Availability 0.6667 -> 1.0 while strict quality 1.0 -> 0.75
headline q5_policy: Reference P4: success 0.9325, mean cost 1.4656, p95 latency 50.0
headline postmortems: 2 incidents replay red -> green; Checkpoint 25 passes
headline tests: 528 tests collected and passed; reader questions: 0; cold reproduction: PASS
```

Fast Local Development Reproduction:

```
make venv && make reproduce-fast && pytest tests/phase2 -q
```

31 Evidence Index

Every primary empirical claim in this report resolves to an immutable JSON pointer and generating command in CI:

Claim ID	Rendered Claim Value	Source JSON File	Exact JSON Pointer	Generating Script	Limitation
C01	Synthesis: Recovery must improve user outcomes	day28/evidence/claim_audit.json	/claims/C01/sources	make	L01, L08
C02	Q1: $p_1 = 0.972$; 8-hop=0.288; naive=0.797; hop 3	day07/evidence/q1_results.json	/curve/7/measured	day28-publication day07/scripts/run_q1.py	L01, L02
C03	Q2: 44 samples, Prec 1.0, Rec 0.615, 10 FN	day15/evidence/q2_results.json	/full_labelled_set/overall_micro/recall	day15/scripts/make_evidence.py	L01, L03
C04	Judge: 24 samples, agree 0.75, $\kappa = 0.50$, 6 FA	day16/evidence/agreement_report.json	/judge_vs_human/cohen_kappa	day16/scripts/make_evidence.py	L01, L04
C05	Q3: Indep $K = 3$ (0.76, 1.9 $\times$); Corr $K = 2$ (0.365)	day19/evidence/retry_sweep.json	/correlated/1/success_rate	day19/scripts/make_evidence.py	L01, L05
C06	Q3 Paired: $K = 3$ Indep 0.76 vs Corr 0.415, $p = 2.69 \times 10^{-16}$	day19/evidence/crossover.json	/paired_correlation_effect/mcnemar/p_value	day19/scripts/make_evidence.py	L01, L05
C07	Q4: Avail 0.667 $\rightarrow$ 1.0; Quality 1.0 $\rightarrow$ 0.75	day21/evidence/availability_quality_comparison.json	/comparison/effects	day21/scripts/make_evidence.py	L01, L06
C08	Q4 Slice: 40 fallback, 30 degraded, det recall 0.667	day21/evidence/degradation_detector.json	/recall/rate	day21/scripts/make_evidence.py	L01, L06
C09	Q5 Bounds: Success ≥ 0.78 , Wrong ≤ 0.02 , Cost ≤ 1.75	day24/evidence/policy_comparison.json	/selection/thresholds	day24/scripts/make_evidence.py	L01, L07
C10	Q5 P4: Success 0.9325, Cost 1.4656, p95 Lat 50.0	day24/evidence/policy_comparison.json	/policy_metrics/P4_selective_guarded	day24/scripts/make_evidence.py	L01, L07
C11	Q5 Paired: P4 vs P1 +0.095 succ, $p = 1.95 \times 10^{-9}$	day24/evidence/paired_statistics.json	/paired_dominance_vs_other_eligible/P1_bounded_retry/success_mcnemar	day24/scripts/make_evidence.py	L01, L07
C12	Q5 Stress: Worse sev 0.695; Tight budget 0.575	day24/evidence/winner_attack.json	/attacks/tight_budgets/metrics/user_visible_correct_success/rate	day24/scripts/make_evidence.py	L01, L07
C13	Cascade synthesis: selective, capped, guarded	day24/evidence/q5_recommendation.json	/tradeoff	day24/scripts/make_evidence.py	L01, L07
C14	Sample sizes: 500 Q1, 44 Q2, 200 Q3, 120 Q4, 400 Q5	day28/evidence/claim_audit.json	/claims/C14/bindings	make	L01–L08
C15	Stats methods: Wilson, McNemar, 10k Bootstrap	day14/evidence/stats_verification.json	/checks	day28-publication day14/scripts/make_evidence.py	L01, L08
P00	Preflight: 6 runs live; temp-0 deterministic	projects/p00_preflight/preflight.json	/live_runs	projects/p00_preflight/run.py	Provisional
P02	OTel Exporter: 350 traces, 5,392 spans, 100% SemConv	projects/p02_otel_exporter/results.json	/conformance/passed	projects/p02_otel_exporter/run.py	Synthetic
P13	SLO Alert: 15 $\times$ fast-burn page; 24m incident drill	projects/p13_slo_incident/results.json	/phases/incident/slos/grounded_pass_rate/burn_rate	projects/p13_slo_incident/run.py	Simulated
P15	Resilience: +76% pass gain at 30% faults; 80% shed	projects/p15_resilience/results.json	/comparisons/2/cost/reliability_gain	projects/p15_resilience/run.py	Mock endpoint
P16	Runtime Policy: 1,000/1,000 hostile denied; 0 FP	projects/p16_runtime_policy/results.json	/adversarial_enforcement/deny_rate	projects/p16_runtime_policy/run.py	Hostile stub

32 Roadmap

The forward roadmap separates immediate engineering obligations from medium-term research:

Horizon	Objective	Deliverable & Methodology	Engineering Value	Cost Cap
Immediate	Execute Preflight Key Probe	Hit R1–R6 once with live API keys to verify pricing and prefix caching.	Unblocks Phase 2 paid sweep ladder.	\$0.05
Milestone 1	Project P1: Real Baseline	Sweep 100 standard scenarios across R2 (Qwen 3.8 Flash).	Proves real model harness without modifying oracle.	\$2.00
Milestone 2	Project P3: Grounding Zero Point	Measure $n = 200$ across R2 and R6; freeze 150 hard scenarios for P6.	Resolves Pre-Registered Hypothesis H1.	\$5.00
Milestone 3	Project P4: Failure Taxonomy	Human qualitative open coding of 150 real failure traces by engineer.	Resolves H2; grounds taxonomy in real LLM errors.	\$0.00
Milestone 4	Project P5: Judge Calibration	Multi-judge Cohen's kappa evaluation on human-coded taxonomy.	Resolves H3; calibrates advisory judges.	\$10.00
Milestone 5	Project P6: Pass ^k Decay Sweep	2,700 runs across R1–R6 on 150 hard scenarios ($k = 3$).	Headline publication resolving H4 and H5.	\$45.00
Medium-Term	MCPTox Bounded Defense (P8)	Test bounded Pydantic agent against MCPTox malicious tool metadata.	Resolves H7; evaluates tool poisoning defense.	\$10.00
Long-Term	Calibrated Cascade (P10)	Train Pareto-optimal routing classifier on P5/P6 empirical data.	Automates cost-optimal escalation.	\$8.00

33 Final Engineering Assessment

33.1 Brutally Honest Staff Evaluation

Writing an honest portfolio requires evaluating one's own work with the same skepticism an external staff reviewer would apply:

33.1.1 What is Genuinely Strong and Unusually Rigorous

- **Causal Scientific Pairing:** Pairing requests on identical master seeds and computing McNemar tests on discordant pairs eliminates scenario noise.
- **Oracle Independence:** Refusing to allow an LLM judge to grade the agent's core success, and proving the judge's blind spot on borderline tokens.
- **Fail-Closed Reproducibility:** The JSON pointer audit binding every displayed number to committed artifacts in CI sets a standard rarely seen in AI research.
- **Zero-Cost Foundation Phase:** Shipping contracts, link-layer corpus, OTel exporter, SRE alerts, resilience engine, and runtime policy with \$0 spend.

33.1.2 What is Ordinary

- **The Agent Architecture:** The single-agent reason-tool-observe loop is simple by design. It uses standard prompt-based tool dispatching without novel agentic patterns.
- **The Tool Suite:** Three tools (`search` , `lookup` , `calc`) over an in-memory document store.

33.1.3 What is Still Weak and Unfinished

- **Unrun Phase 2 Live Sweeps:** The model ladder remains provisional. No paid sweeps across commercial LLM APIs have executed; \$0.00 has been spent.
- **Small Labelled Sample Sizes:** The Q2 detector evaluation ($n = 44$) and Day 16 judge evaluation ($n = 24$) have wide confidence intervals.

33.2 Hiring Manager Inference Boundary

WHAT A HIRING MANAGER CAN INFER	WHAT A HIRING MANAGER CANNOT INFER
• The author understands distributed systems failure modes (retry storms, fallback degradation, cascades). • The author possesses world-class measurement discipline (Wilson intervals, paired bootstraps, MEC contracts). • The author writes production-grade, crash-resilient infrastructure (SQLite spans, OTel SemConv, Pydantic v2). • The author has the intellectual integrity to disclose failures (Amendment A-002, LLM judge quarantine).	• That the author has solved universal LLM hallucination or fine-tuned frontier models. • That these exact recovery parameters ($K = 2$, \$1.46 cost) apply directly to an uncalibrated enterprise production system. • That the author has executed large-scale multi-thousand-dollar model sweeps (which remain pending in Phase 2). • That synthetic document-grounded benchmarks fully capture the nuances of human conversational ambiguity.

Concluding Engineering Assertion

“FAULTLINE establishes that reliability engineering in generative AI is not a matter of prompting or faith. It is a measurement discipline: define user-visible correctness, isolate failure modes, pair experimental observations, enforce explicit budget ceilings, and publish the boundary where the recommendation stops working.”